Ref: 2020/ II3 / ..... Defended in the July session

University of Manouba
National School of Computer Science

Graduation Project Report

Submitted in purpose to obtain

The National Engineering Diploma in Computer Science

Made by

Chaima Ben Zouitine

Topic : Driver scoring application

Host comapny : **Machinestalk**

machinestalk
innovation frontier

Responsible Name : **Mr Nezih Dammak**

Internship Supervisor : **Mr Meher Benahmed**

University Tutor : **Mrs Raoudha Chebil**

Adress : **Lloyd building- 9th floor , centre Urbain Nord**

Tel : **24898467**

Signatures

MR MEHER BENAHMED
(Machinestalk)

I sincerely appreciate the time and effort spent by Chaima in moving the project to a new level. his behavior and conduct has had a positive impact on a number of member I appreciate her engagement and enthusiasm and her ability to self motivate others has resulted in significant increase in the company's productivity and profitability

machinestalk
innovation frontier
MACHINESTALK TUNISIA
Immeuble Zaineb 9A898 Centre Urbain Nord 1003, T
MF : 1303197W/A/M/000

MRS RAOUDHA CHEBIL
(ENSI)

Dedicaces

I dedicate this work with gratitude to all the persons I love:

To my mother for her unconditionally love and support

To my family

To all my true friends

Chaima Benzouitine

Acknowledgment

THe timely and successful completion of this project could be hardly achievable without the assistance of many people. I will take this opportunity to express my deep gratitude to them.

I would like to express my sincere gratitude to my academic supervisor **Mrs Raoudha CHEBIL** for her availability, support, regular monitoring, quality supervision and her precious advice throughout the realization of this project. Her assistance is much appreciated.

I wish to extend my gratitude to my professional supervisor **Mr. Meher BENAHMED**, a big data engineer at Machinestlk, for sharing his pearls of wisdom throughout the internship. I am immensely thankful for his valuable guidance and constant encouragements.

A special acknowledgement goes to the team of Machinestalk for ensuring a smooth running of my internship. I consider it an honor to work with them and to learn from them.

A special acknowledgement goes to **Sarra HAMMAMI**, a grahic designer at Machinestlak, and **Yahya IBRAHMI**, a devops engineer, for supporting me and sharing with me their expertise.

Contents

General introduction **1**

1 General Context **2**
1.1 Company overview . . . 2
1.1.1 Machinestalk company . . . 2
1.1.2 Mission and vision statements . . . 2
1.1.3 Machinestalk values . . . 2
1.1.4 Machinestalk Projects . . . 3
1.2 Project Context . . . 3
1.2.1 Problem Description . . . 3
1.2.2 Suggested Solution . . . 4
1.3 Development Methodology . . . 4
1.3.1 Agile methodology . . . 5
1.3.2 Scrum framework . . . 5
1.3.3 DevOps approach . . . 7

2 Preliminary Study **9**
2.1 Basic concepts . . . 9
2.1.1 Big Data . . . 9
2.1.1.1 Data explosion and types of data . . . 9
2.1.1.2 The need for big data . . . 9
2.1.1.3 Characteristics of big data . . . 10
2.1.1.4 Big Data technology capabilities . . . 10
2.1.2 Stream Processing . . . 10
2.1.2.1 Definition . . . 10
2.1.2.2 Fundamental stream processing aspects . . . 10
2.1.2.3 Data stream processing tools . . . 11
2.1.3 Big data analytics . . . 13
2.1.3.1 Definition . . . 13
2.1.3.2 Lifecycle of big data analytics . . . 13
2.1.3.3 Types of big data analytics . . . 14
2.2 Study of existing solutions . . . 14
2.2.1 Example of applications . . . 15
2.2.1.1 DriveSmart . . . 15
2.2.1.2 Aviva Drive . . . 16
2.2.1.3 TomTom Curfer . . . 16
2.2.1.4 Driver Performance Scoring Application . . . 17
2.2.2 Criticism of the existing . . . 18

3 Specifications and Design **19**
3.1 Requirement analysis . . . 19
3.1.1 Actors identification . . . 19
3.1.2 Functional Requirements . . . 19
3.1.3 Non-Functional Requirements . . . 20
3.1.4 UML diagrams . . . 20
3.1.4.1 Use case diagram . . . 20
3.1.4.2 System sequence diagrams . . . 22
3.2 Global Design . . . 24
3.2.1 Logical architecture . . . 24
3.2.1.1 Lambda architecture . . . 24
3.2.1.2 Kappa architecture . . . 25
3.2.1.3 Adopted architecture . . . 26
3.2.2 Physical architecture . . . 26
3.2.3 Application flow process . . . 27
3.3 Product Backlog and Sprints planning . . . 28
3.3.1 Product Backlog . . . 28
3.3.2 Sprints planning . . . 29

4 Achievement **31**
4.1 Hardware Environment . . . 31
4.2 Software Environment . . . 31
4.2.1 Big Data tools . . . 31
4.2.1.1 Apache Flink . . . 31
4.2.1.2 Kafka . . . 32
4.2.2 Management and Development tools . . . 33
4.2.2.1 Bitbucket . . . 33
4.2.2.2 Git . . . 33
4.2.2.3 Jira . . . 34
4.2.3 Data analysis tool . . . 34
4.2.4 Programming language . . . 34
4.2.5 DevOps tools . . . 35
4.2.5.1 Docker . . . 35
4.2.5.2 Jenkins . . . 35
4.3 Features Release . . . 35
4.3.1 Sprint 1 : Data Pre-processing . . . 35
4.3.2 Sprint 2 : Driver Score Algorithm . . . 37
4.3.2.1 Stage one . . . 38
4.3.2.2 Stage Two . . . 39
4.3.3 Sprint 3 : Driver score implementation . . . 41
4.3.3.1 Speed limit determination . . . 41
4.3.3.2 Apache flink windows mechanism selection . . . 42
4.3.3.3 Ingestion and storage of data . . . 43
4.3.3.4 Screen shots . . . 43
4.3.4 Sprint 4: Data analysis . . . 45
4.3.5 Sprint 5: Application deployment . . . 47

General Conclusion **49**

Glossary

IoT *Internet of Things*

API *Application Programming Interface*

UML *Unified Modeling Language*

CPU *Central Processing Unit*

RAM *Random Access Memory*

GPU *Graphics Processing Unit*

OOP *Object Oriented Programming*

FP *Functional Programming*

SQL *Structured Query Language*

FMS *Feet Management System*

CI *Continuous Integration*

CD *Continuous Deployment*

HTTPs *HyperText Transfer Protocol Secure*

DAG *Directed Acyclic Graph*

RDD *Resilient Distributed Dataset*

GPS *Global Positioning System*

IT *Information Technology*

General Introduction

Actually internet of things (IoT) is increasingly applied in a growing number of industries working in several fields, transportation included, accompanied the evolution of the modern world. The transportation field particularly is the second largest sector investing in the Industrial Internet of Things (IIoT), spending approximately $78 billion from 2016 [1]. Thanks to Iot, we are witnessing thoroughgoing changes and improvements in transportation that promote man life with maximum safety and confort. However, even with tremendous success and innovations seen in transportation field, car accidents are listed among the top ten leading causes of death across the world. As the driver behavior strongly affects the road security, special emerging applications for drivers tracking have been suggested to tackle this issue. Such applications shown particular interest specially in the fleet management and car insurance. They allowed to car insurers to introduce a new insuring pattern named "pay how you drive" and gave to fleet owners have the opportunity to survey the behavior of their drivers.
Driver behavior profiling is a part of interests of the comapny Machinestalk which hosted our internship. This topic would be an effective service supplemented their project to improve the behavior of drivers and leveraging a safer and right aware driving.
The task of this internship is taking part of developed projects of Machinestlak which generally aim to fleet management and tracking. The main objective of the project is to develop a generic driving score module allowing to better survey driver behavior and performance.

This report is organized around five chapters that describe the progress of the project. In the first chapter, we start by introducing the general context of the project, namely the host organization and the context of the project.

The second chapter includes a theoretical study which defines the most interesting and essential concepts and notions for the project as well as a study on existing solutions. Then we specify the needs of our application in the third chapter based on UML diagrams. The conceptual part of the project will be described in the fourth chapter which includes the overall design and the detailed design.

Finally we will end with detailing the stack of technology used as well as the results obtained.

Chapter 1

General Context

In this chapter, we situate in place our project in its general context. First, it starts with the presentation of the host company where we had the opportunity to work. Then we briefly describe the project context by presenting the problem statement and the suggested solution and finally the work methodology adopted to get the job done.

1.1 Company overview

Our project was held in Machinestalk. In this section, we present the main activities of this company.

1.1.1 Machinestalk company

Active for more than 10 years, Machinestalk, NOMD subsidiary, is a business service company that delivers IoT solutions to the world's global product manufacturers. Machinestalk's head office is in Saudi Arabia. It operates in more than four countries, including Tunisia. By combining the expertise of its team and the power of technology, Machinestlk considered as a regional reference of IoT solutions. It paves the way for its customers to have cutting edge various solutions in order to provide them with the insights and powers to improve their lives [2].

1.1.2 Mission and vision statements

- **Mission**: Machinestalk strives to stay at the forefront and becoming a regional reference of IoT solutions[2]. The company pledge customers with cutting-edge diversified solutions into commodity, comfort and the valuable insight that they offer.
- **Vision** : Machinestalk's vision is to be the backbone of the IoT regional market and the dependable partner for whom interested in this domain[2].

1.1.3 Machinestalk values

What helped Machinestalk shaping such success are its following characteristics [2] :

- **Competent** : having sufficient skills to be leader in the domain.

- **Customers-centric** : ensuring clients satisfaction and leaving a positive and everlasting impact.
- **Innovative** : delivering ever ingenious and excellent products.

1.1.4 Machinestalk Projects

Machinestalk has investigated in numerous fields of Iot such as transportation and logistic, smart city and industry[2]. In the following we will present the projects related to our work :

- **Fleet management systems :** FMS provides a set of services that concern every fleet manager including all kinds of transportation assets such as cars, trucks and boats. FMS allows customers to manage, monitor and track company vehicles anytime and anywhere, as well as offer extra facilities like alerts when vehicles cross certain areas or exceed limits. Theses services can also be used to monitor maintenance, check schedules and present reports on vehicle activity and status. FMS is a beneficial suite of services aiming to maximize daily working efficiency and guarantee safety.
- **Car sharing :** Car sharing platform provides continuous monitoring of the car fleet and a rich set of options in order to improve the users' convenience.
  All via one platform,the monitoring process can be defined starting with unlocking and turning the vehicles on, through locating their position and tracking their movement, down to identifying battery levels and number of kilometers moved for each vehicle.
- **Motorna :** is a mobile application that's main goal is to achieve safety during a trip. Motorna enhances the tracking of the vehicle by offering a range variety of services including trips history, live tracking, vehicle diagnosis, live notifications and WIFI accessibility.

1.2 Project Context

In this section, we present the project context by exposing the problem and the proposed solution statements.

1.2.1 Problem Description

Machinestalk, being a leading IoT-based solutions provider, has been concerned to tackle transportation related issues. This field is considered highly risky as accidents and incidents may take place at any time. However, travelers expect transportation services to be safe. Businesses who hire drivers for their employees are highly concerned with their security and welfare. Vehicle leasing companies want to serve their clients with an outstanding and safe service with lower costs. Parents who hire Driver for their kids want to guarantee safe pickups and drop offs for them. Generally, companies that rely on vehicles are increasingly looking for ways to eco drive and saving money is their number one priority. In this context, machinestalk has been working on cutting edge solutions to deliver its clients with an effective fleet management service

by developing several applications such as Motorna, FMS that deliver a wide range of services that aim at improving the safety and the security level within the transportation field. These platforms offer to their users a set of insights that include the vehicle diagnosis as well as live tracking to track the driver and identify its position in real-time. But driver geographic monitoring is not enough to solve given postures. What is really important is to spot continuously how drivers act behind the wheel since the ability of a driver to control his vehicle is essential for driving safely. It is exactly this constatation which motivated the work on the solution described in the following part.

1.2.2 Suggested Solution

It is evident that in order to ensure safety to our roads, a courteous and safe driving habits have to be exhibited. The aim of this project is to shed the light on the driving behavior since it is considered as a primary reason behind the quality of the vehicle path-following. We consider that monitoring and scoring the driver behavior would help us ensure the safety of the passengers and reduce the risk of road trauma. These fleet-related business challenges place the quality of driving behavior at the center of attention. It allows to both control expenses and ensure a safe traveling on the road.
If bad driving is detected, drivers can be retrained in economical driving or businesses can take precautionary measures.
Speed, hard braking, rapid acceleration and other factors are the types of scenarios that mainly affect the way of driving and should be eliminated. Aggregating these relevant records captured during a road into an instantaneous score which reflects the behavior of the driver. Thus, keeping an eye on driver behavior enables, on one hand, fleet managers to monitor the most at risk drivers and improve the overall safety of their business and on the other hand, the driver to be up to date with his own driving style to avoid bad habits and enhance his driving behavior.
So, we propose to develop a real-time driving score module. It is the process of automatically gathering data from device installed into vehicle and applying a computational pattern to them for the purpose of generating the driver score.

1.3 Development Methodology

In order to achieve quality software that meets the needs required in time and predictable costs. Choosing the right development process is a essential phase in the life cycle of the project. By analyzing the existing factors, we we split the project into sundry iterations. At each iteration a part of the project will be treated. At the end of each iteration, an autonomous microservice will be delivered. To do this, we have chosen to adopt the Agile SCRUM method for development and the DevOps approach for production. Indeed, as shown in the diagram of the figure 1.1, agility affects all stages of the project and not just the teams of development.

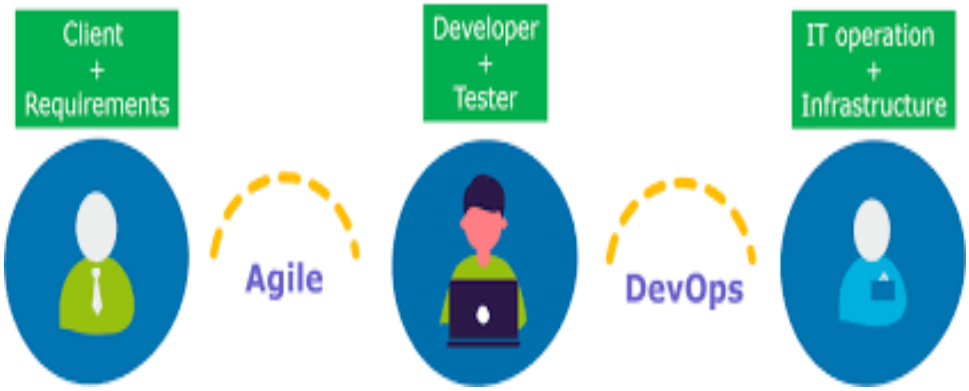


Figure 1.1: End-to-end agility [7]

1.3.1 Agile methodology

Back in the 90's, IT projects were managed by the waterfall methodology where the executed tasks cascaded from analyzing the requirements to designing , implementing , testing and roll-out.
Although the approach was exceedingly structured and organized, it was suffering from many shortcomings. The most important one is that by the time the project was delivered to the customers requirements can change or wrong interpreted which was expensive cost and unsatisfying for the clients.That was the case until 2011, a group of 17 software practitioners come up with the Agile Manifesto which is a new mindset focusing on four values and twelve principles [8].
These values are :

1. Individuals and interactions over process and tools;
2. Working software over comprehensive documentation;
3. Customer collaboration over contract negotiation ;
4. Responding to change over following a plan.

For the principles, we mention customer satisfaction, collaboration, embracing changes, empowerment, continuous improvement, operational excellence, self-organization, simplicity, good metrics, steadiness, effective communication and speed delivery [8].

1.3.2 Scrum framework

Introduced at OOPLSA in 1995 by Ken Schwaber and Jeff Stherland, Scrum is a lightweight process framework and simple to understand for developing, delivering and sustaining complex products.
Because scrum is highly compatible with the agile values and principles,it is considered as a subset of Agile and it's been in wide use for over 20 years in a variety of industries [9]. It's a very flexible development strategy when a team works together to

achieve a common goal.
Scrum is based on breaking up the project into iterations called "Sprint". A Sprint can take a duration which generally varies between two weeks and a month. Before each Sprint the tasks are estimated in time and complexity. The figure 1.2 describes the scrum process.

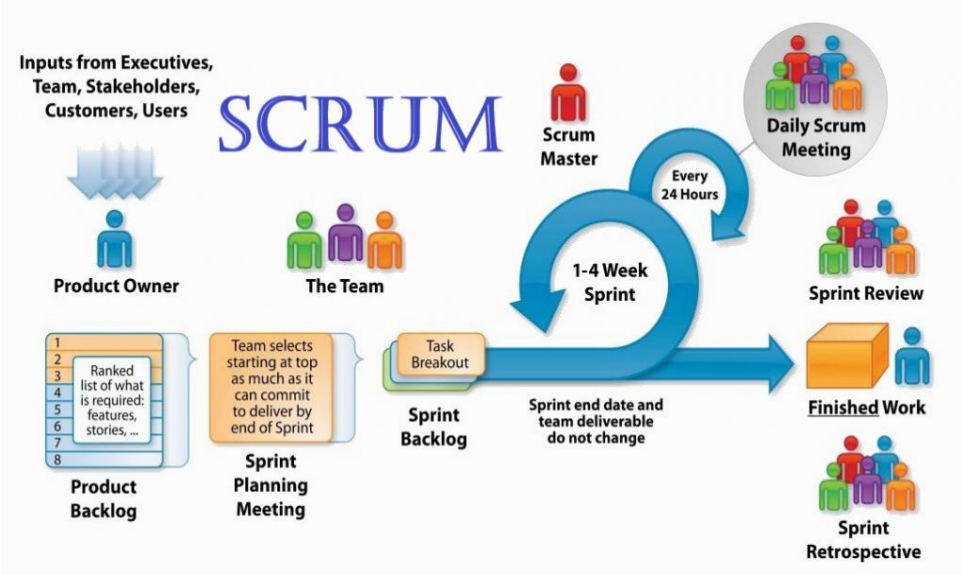


Figure 1.2: Scrum process [9]

The SCRUM framework defines three roles [9] for a project as shown in the figure 1.2 :

- **Product owner :** he is the official representative of the client within of a scrum project. It defines and prioritizes product needs. This role is played in our project by Mr Nezih Dammak, the technical director of Machinestalk.
- **Scrum master :** he is the person who facilitates the implementation of the method and ensures compliance with its objectives. He is responsible for communication. He does not play the role of a project manager, but he is responsible for removing any impediments that would prevent the advancement of the team and the project during the various Sprints.
- **Scrum team members :** these are the people responsible for realizing the sprints and the deliverables. Members must collaborate together and be autonomous to meet customer requirements and established commitments.

Several meetings, illustrated in the figure 1.2, are implemented to allow communication between the actors presented [9]. We cite among them :

- **Sprint planning meeting :** it is a critical and important meeting in the life cycle of the project, and takes place at the beginning of Sprint. It allows the project team and the product manager to define all of the goals to be achieved during the Sprint based on the prioritized "Product Backlog" and estimate the production capacity needed to achieve this goal.

- **Daily Scrum :** Once a day, project team members spend an average of 15 minutes together talking. Each participant reports what they did the day before and the obstacles they encountered. We also discuss the tasks to be carried out on the day of the meeting.

- **Sprint Review Meeting :** At the end of the Sprint, the project team presents the functionality achieved during the Sprint and collects feedback from the Product Owner and end users.

- **Sprint Retrospective Meeting :** Another meeting that takes place at the end of each Sprint. The SCRUM Master moderates this meeting. The project team judges their behavior and performance in the development process to take measures to better manage the next Sprints.

Communication artifacts are also part of the development life cycle of a project following the SCRUM methodology as shown in figure 1.2. Among these artifacts, we mention :

- **Product Backlog :** It contains the list of functionalities desired by the customer in the form of User Stories. It is visible to all stakeholders who can add elements. The priorities of the elements are constantly updated by the Product Owner.

- **Sprint Backlog :** It includes requirements to be negotiated between the product manager and the project team when planning the Sprint. It is visible to the entire team that it uses as a reference in daily scrums.

Agile methods represent an innovative movement that aims to bring more value to customers and users. However, they are only applied to developments and are hampered during deployments. The DevOps approach aims to lift this blockage.

1.3.3 DevOps approach

DevOps, a mantle word between "Development" and "Operations", is a software development strategy not a tool neither a framework , it is an approach that aims to bridge the gap between development team and the operations team. By simplifying the relationships between these two actors, it becomes easier to quickly iterate the phases of deployment while increasing reliability[3].
The DevOps approach is based on four strands defined as the CAMS model [3] as illustrated in the figure 1.3. This model accounts for Culture, Automation, Measurement and Sharing :

- **C as Culture :** It is essential to the success of the DevOps team. It is based on the collaboration in which all the people involved in the project work together and use a standardized set of values, tools and workflows and follow a common objective to guarantee efficiency in terms of management or troubleshooting.

- **A as Automation :** Automation helps create more reliable products faster and reduce manual work. It can be resumed in three words : continuous integration, continuous delivery and continuous deployment.

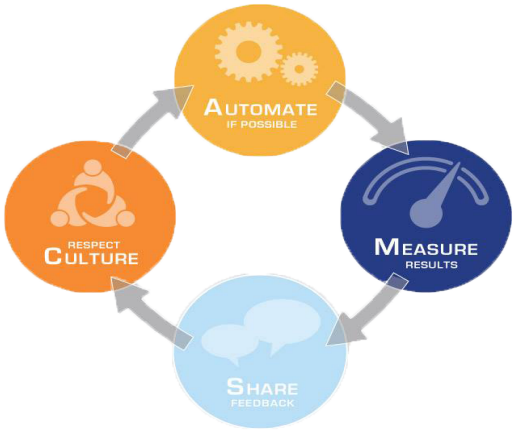


Figure 1.3: CAMS model [3]

  - Continuous integration is a set of practices aimed at improving the quality of a application. It usually refers to integration activities, build and test code in the development environment.
  - Continuous delivery is a development discipline in which we let's build software that can be deployed in the production environment anytime.
  - Continuous deployment means that any change is automatically gone into production.

- **M as Measurement :** Based on Lean and Agile approaches, DevOps practices promote continuous improvement and the ability to anticipate failures, allowing teams to learn constantly in order to optimize the product and improve the quality of work.

- **S as Sharing :** Sharing is the foundation of DevOps. Culture, automation and measurement rely on this pillar. This revolutionary change in approach means that teams share their knowledge, tools, flows, information and goals to create better digital products faster.

Conclusion

In this chapter, we presented first the hosting society, then the context of the subject through the scope and objective of our work as well as the adopted methodology of work. In sight to follow a logical progression in this report, a theoretical study concerning the basic concepts of the project and the existing solutions will be the subject of the next chapter.

Chapter 2

Preliminary Study

Throughout this chapter, we introduce the concepts necessary for the development of the global solution.Then we analyze the solutions already developed and identify their limits in order to overcome them.

2.1 Basic concepts

In this section, we present the key elements of our application. We shed the light on big data field, then we discuss the stream processing and we finalize with the big data analytics.

2.1.1 Big Data

2.1.1.1 Data explosion and types of data

Over the last decade, there has been an unimaginable explosion of data in every sector. By an estimate, over 2.5 exabytes of data is generated every day, equivalent to 2.5 billion gigabytes [4]. This huge volume of data originates from various sources such as social networks which generate daily more than 10 terabytes of data, sensor networks which send continuously data as well as web logs, GPS systems and astronomy and medical records [4]. Data can be identified as one of three primary types. The first is structured data, which can be represented in a tabular format and stored in a relational databases.The second is semi-structured data, which does not have a formal data model, though it may have some semblance of a structure.The third is unstructured data, which does not have a predefined data model. Text files, and weblogs are examples of unstructured data.

2.1.1.2 The need for big data

By an estimate, around 90 percent of the world's data has been created in the last two years. Moreover, 80 percent of the data is unstructured or available in widely varying structures, which are difficult to analyze[4]. Through the years of developing I.T systems, we have observed that structured formats like databases have some limitations with respect to handling large quantities of data. It also has been observed that it is difficult to integrate information distributed across multiple systems. Further, most business users do not know what should be analyzed and discover requirements only

during the development of I.T. systems[11]. As the data has grown, so have data lakes within companies, and subsequently potentially valuable data that has been generated from various systems such as ERP or CRM, and supply chain management which is dormant, either thrown away. As result, it become too expensive to justify the integration of large volumes of data [11].

2.1.1.3 Characteristics of big data

Big Data is mainly characterized by the following aspects often called the 3 Vs [12] :

- Volume : The amount of data to be processed is incessantly growing;
- Velocity : The speed at which data is generated, captured and shared is very elevated;
- Variety : The data are in diverse formats and not necessarily structured.

Beside, new Vs are defined.They may be summarized as the Vision we have about big data, the Value big data carries, the Visualisation tools to use or the Variability in the consistency of big data [12].

2.1.1.4 Big Data technology capabilities

Big data technologies became very popular because it helps to process and manage a huge amount of data in a cost-efficient manner [5]. Furthermore, it analyzes all available data in their native forms which can be structured, unstructured or streaming. Added to that, Big data technology is interesting due to its ability to capture data from fast happening events in real time and turn it into actionable insights.

Big data technologies help to understand and navigate big data sources, manage and store a huge volume of a varied data and processed it in reasonable time [5].

2.1.2 Stream Processing

2.1.2.1 Definition

Stream processing, known also as real-time analytics,complex event processing or real-time streaming analytics, is considered among big data technology [13]. It's applied to query continuous data and detect patterns over time as it's being generated. Big data establishes the value of insights derived from processing data. These insights have not the same value. Some of them have much higher values shortly after something has happened and that value diminishes very fast with time. Stream processing targets such scenarios [13]. The key strength of stream processing is that it can provide insights faster, often within milliseconds to seconds.

2.1.2.2 Fundamental stream processing aspects

The following important features are associated with flow processing. They allow to determine the strengths and limitations of any streaming framework [6] :

- **Delivery Guarantees :** is a delivery mechanism that, whatever happens, a particular incoming record in a streaming engine will be processed. It can be treated either at least once, at most once or exactly once even in case of failure.
- **Fault Tolerance :** fault relates to a failure, so fault tolerance is the framework's ability to handle and recover the loss after a fault and should start processing again from the failure point.
- **State Management :** in the event that we require a stateful processing where we need to define a certain state,the framework should be able to provide mechanisms to permanently maintain and update the state of the records.
- **Performance :** it comprises latency which defines how quickly a record can be processed, the throughput of records processed per second and scalability. Preferably, the latency indicator should be as lower as possible wheras the throughput should be higher
- **Advanced Features :** if stream processing needs require a complex treatments we will need an advanced features to handle them such as watermarks , windowing , event time processing ...
- **Maturity :** is the framework's ability to be proven and tested on a large scale by large companies and more likely to gain good support from the community.

2.1.2.3 Data stream processing tools

Recently, many stream processing frameworks oriented big data have emerged. Among the main open-source engines, we cite Storm, Spark and Flink.

- **Storm**: is a real-time stream processing, it is a part of Apache projects originally created by Nathan Marz and the BackType team in September, 2011. Written mainly in the Clojure programming language, Storm has been designed to process unbounded tuple streams in a distributed manner [14].
  In Storm, the tuples are generated by spouts and the treatments are carried out by bolts. The transmission of data from spouts to bolts can be represented, as described in the figure 2.1, in the form of graphs whose nodes are spouts and bolts and the connections between the nodes are flows [14]. the entire topology forms a DAG(directed acyclic graph).

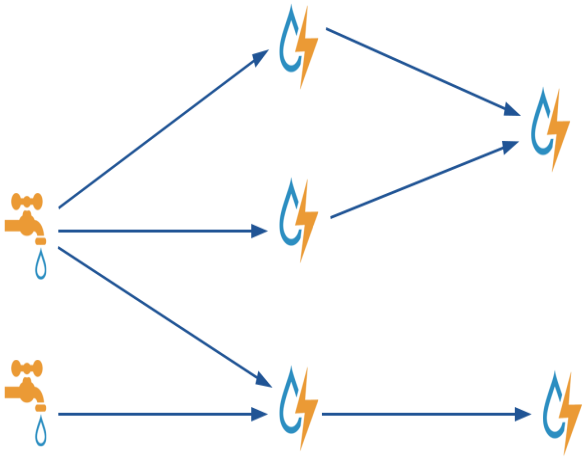


Figure 2.1: Apache Storm Architecture [14]

- **Spark** : started in 2009 as a project at UC Berkley AMPLab and in 2013 it became an Apache top level project. Built based on hadoop, Spark is an open source in-memory computing framework which is used to process data in batch or near to real-time across various clusters of computers[16]. The architecture of Spark is based mainly on Resilient Distributed Datasets (RDDs) which represent a collection of items distributed through many compute nodes that can be manipulated in parallel [15].
  Spark follows the master slave pattern [16]. According to the figure 2.2, when we launch a treatment on the Spark framework, we go through the Driver which is in a way the master who communicates with the cluster manager. The latter manages the resources of the workers who execute the requested processing.

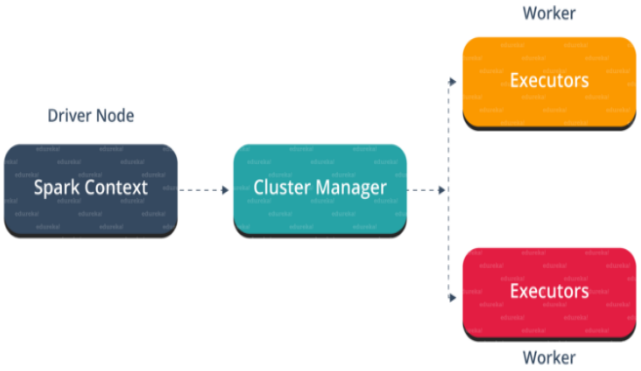


Figure 2.2: Apache Spark Architecture [16]

- **Flink** : Started in 2009 at Berlin TU University, Flink became one of the top level project of the Apache software foundation in December, 2014. It is actively developed by data artisans and other contributors [17].
  Flink is an open source powerful engine for streaming processing that also can handles batch, Graph and in-memory processing with lightning speed, easy of use and sophisticated analytics [17]. It is rich sets of libraries, its APIs make it easy to model complex real world business problems and allow to strengthen its possibilities in Big Data analysis and Machine Learning.
  The diagram 2.3 summarizes the ecosystem of Flink.

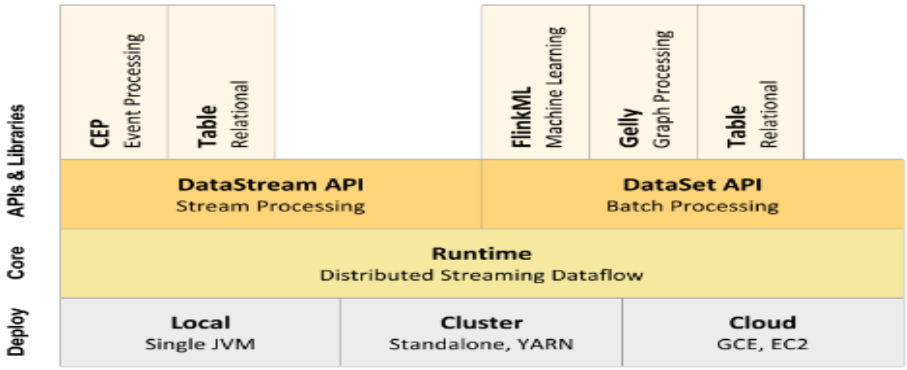


Figure 2.3: Apache Flink Ecosystem [18]

2.1.3 Big data analytics

2.1.3.1 Definition

Big data analytics is defined as the process used to extract meaningful informations that aims to identify the most interesting aspects of their structure and drawing conclusions [19].

2.1.3.2 Lifecycle of big data analytics

As shwon in the figure 2.4, the big data analytics starts with the business case evaluation stage which define the motive behind the analysis. Once this is done, it becomes much easier to proceed with rest after which we look into the various data sources from where data are gathered which are required for analysis. Once the data is received, it will pass through a filtering stage where all the corrupt data will be removed. Next, the incompatible data with the tool that used in analysis is extracted and transformed to a compatible form. In the next stage, known as the data aggregation stage, all data with the same fields will be integrated. Then, we pass through the analysis phase, it is a very important phase in the lifecyle of the big data analytics. In this step, the entire process of evaluating data using various analytical and statistical tools to discover meaningful information. The result of the data analysis stage is then graphically communicated.

Hence, the final analysis result is made available to business stakeholders for making decisions [20].

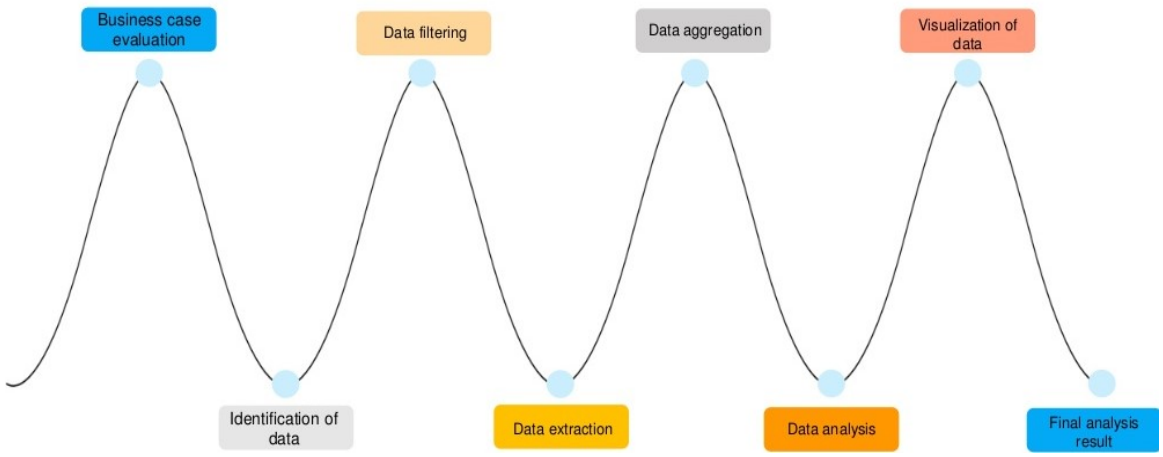


Figure 2.4: Lifecycle of big data analytics [20]

2.1.3.3 Types of big data analytics

Big data analytics is defined by its following four types :

- **Descriptive analytics :** the conventional form of BI, is the type that summarizes past data into an interpretable form [21]. It helps to analyse the incoming data, which had occurred at any time that could be a year, week or even minute ago,for insights. It is more reactive in nature and simply replies to the "what has happened" question.
- **Diagnostic analytics :** asks the question why a particular problem has occured. It allows a deeper analysis to discover the root cause of the problem [21].
- **Predictive analytics :** this type helps to make predictions of the future. It analyzes the current and the historical facts and applied a machine learning model to forecast what will happen next [21].
- **Prescriptive analytics :** this type prescribe a solution for such problem. It uplift the predictive analytics to the next stage. Since, an idea about what will happen in the future is built then the question is what is the solution [21]. Hence, a set of actions is suggested and their potential consequences.

2.2 Study of existing solutions

In this part, we present a study and then a criticism of the existing solutions for the stated problem. Thereafter, the solution envisaged is succinctly presented.

2.2.1 Example of applications

2.2.1.1 DriveSmart

The DriverSmart is a mobile application that allows the user to get an overview on the way how he drives. It started by pressing the start button to record the trip. After reaching the target destination, the application provides the driver with details about the duration, distance driven of the trip and the events based on acceleration, braking, speed and turns. Furthermore, this application concludes the journey with a score and personalized advises so that the driver can improve his driving [22].

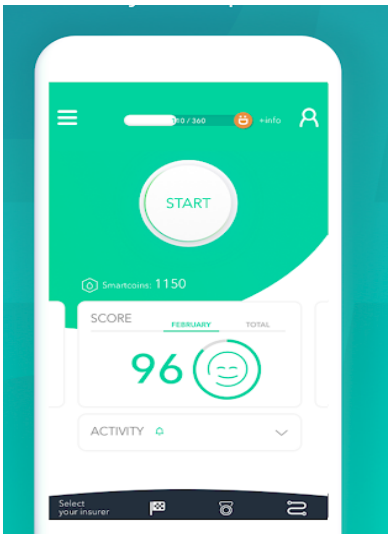


Figure 2.5: DriveSmart interface [22]

2.2.1.2 Aviva Drive

This application works only on smartphones, it aims to track the driving routes, duration of the trip, events such as acceleration and braking, the fuel consumption and more, collecting all this data a driving safety score after reaching 200 miles. The score is bounded between 1 and 10. This application comes essentially for the car insurance discount. When the driver get a score of 3.1 or above he will get a discount. An average of £170 can be saved on their Aviva car insurance. But according to users comments, this application is not stabe [23].

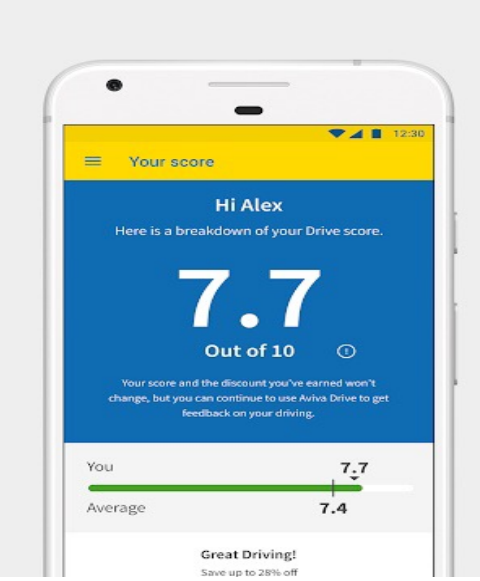


Figure 2.6: Aviva Drive interface [23]

2.2.1.3 TomTom Curfer

This application used to sold by 59 euro and it's no more available. Tomtom CURFER is a smart solution for car enthusiasts who like to be connected to their car and appreciate having information about their driving style.It is a kind of small accessory that plugs into the diagnostic socket of a vehicle that provide real time information on driver driving technique. The application assesses in real time the acceleration, braking, cornering and stopping management events. By gathering this informations, a score out 100 is calculated and color scale from red to green is displayed to visualize driver performance[24] .



Figure 2.7: TomTom Curfer Accessories [24]

2.2.1.4 Driver Performance Scoring Application

The Driver Performance Scoring Application owned by Blue Tree Systems Ltd is considered as one of the most inclusive driver performance application in the industry, it supports only the company account. The driver scoring is depending on three critical aspects of a driver's behavior that have influence on any fleet which are performance, compliance and safety. According to the score, if the value is high in the three mentioned categories, then the driver is categorized as a good driver and it will reap many profits for the business comprising of notably reduced fuel, the costs of insurance and maintenance [25].

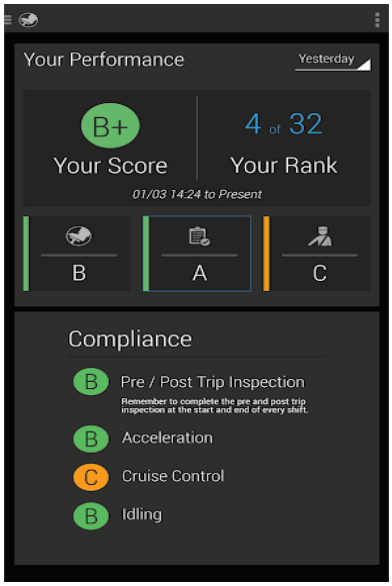


Figure 2.8: Driver Performance Scoring interface [25]

2.2.2 Criticism of the existing

In the previous section, we presented the most popular examples of the driver score applications existing in the market. Some of them are classified as the best applications considering their functionalities, performance, adaptability. But they suffer from many shortcoming. In fact, some of them do not support the real-time aspect or they are privately owned. Besides, each application has its own number of criteria to establish the scorecard of the driver so the score calculation differs from one application to another according to the context needs.
And according to user reviews, some applications are slow and crash all the time. Furthermore, they affirm that certain applications are not stable and returns inaccurate results. Users also complain about the high pricing costs and the lack of functionalities of several systems.
The table 2.1 present a comparison between the different mentioned solutions.

	DriveSmart	Aviva Drive	Tomtom curfer	Driver performance scoring
Client satisfaction	✓	✓	×	✓
Real-time scoring	×	×	✓	×
Public	✓	✓	✓	×
Stability	×	×	×	×
Pricing	×	✓	✓	✓

Table 2.1: Comparison of existing solutions

Conclusion

During this chapter,we made a brief study of the existing solutions we have criticize it.Then we presented the basic concepts that we manipulated during this internship. In the next chapter, we will identify the functionalities of our solution.

Chapter 3

Specifications and Design

Introduction

The purpose of this chapter is to specify, first, the features that our solution offers. Then, we expose the architecture as well as the the design of our work.

3.1 Requirement analysis

In this section, we are interested in requirement analysis. We start by identifying the actors in the system, then we cite the functional and non-functional needs of our application.

3.1.1 Actors identification

We distinguish two main actors for our application:

- ✓ **Fleet manager :** the person who will manage the fleet
- ✓ **Driver :** the person who gets behind the wheel and drive the vehicle.

3.1.2 Functional Requirements

The application must allow to the fleet manager to :

- ✓ **Display the driver real-time score :** the application must inform the fleet manager by the behavior of the driver through exposing a map containing a score icon that display the current score earned by the driver.
- ✓ **Show real-time alerts :** the application must display the real-time alerts and harsh events made by the driver.
- ✓ **Consult driver profiling** : the application must provide to the fleet manager an interface from which he consult the profiling of the driver and offer an overview about trips made by them accompanied by information about these trips.
- ✓ **Evaluate drivers performance :** the application must allow the fleet owner to have a complete vision and insights about driving behavior feedback across analysis and dashboards taken place.

For the driver, the application must satisfy the following requirements :

- ✓ **Display the realtime score :** a score icon will be available by the application to the driver to be up to date with the way he drives during a trip.
- ✓ **Display realtime alerts :** the driver must receive realtime alerts based on occured harsh events so that he can improve his driving style and reduce road risks.
- ✓ **Consult driver profiling and history :** the application allows the driver to have an history about each trip traveled containing the score, the driven distance, the time of the trip and type and number of harsh events.

3.1.3 Non-Functional Requirements

Non-functional needs correspond to the technical requirements and constraints that our system must meet. Our system must necessarily ensure these criteria :

- ✓ **Performance :** the application should be available as much as possible and should ensure reasonable response time regardless of the volume of processed data .
- ✓ **Extensibility :** the system must be able to adapt to needes in the future. Adding other data sources or other metrics for example should be easy.
- ✓ **Reliability :** the application must be reliably safe by providing correct calculations and displaying precise results.
- ✓ **Availability :** unless the device is not working, the application shall be available for user at any moment.
- ✓ **Usability :** the application shall be a generic module that can be easily reused and integrated in different systems.

3.1.4 UML diagrams

In the following, use case diagram and system sequence diagrams of our application are presented.

3.1.4.1 Use case diagram

According to figure 3.1, the fleet manager must be able through the system to track his fleet and analyze how his drivers behave. A scorecard will be available for the manager including a performance history as well as reports of drivers with the intention that he evaluates them and pinpoint risky ones. He will be continuously informed by driver score and the occurred harsh events.
For the driver, by analyzing every captured data from the device, he will keep track of the way he acts behind the wheel. It is a kind of coaching program that allow him to improve the way he drives by showing its real-time score reaped and real-time alerts.

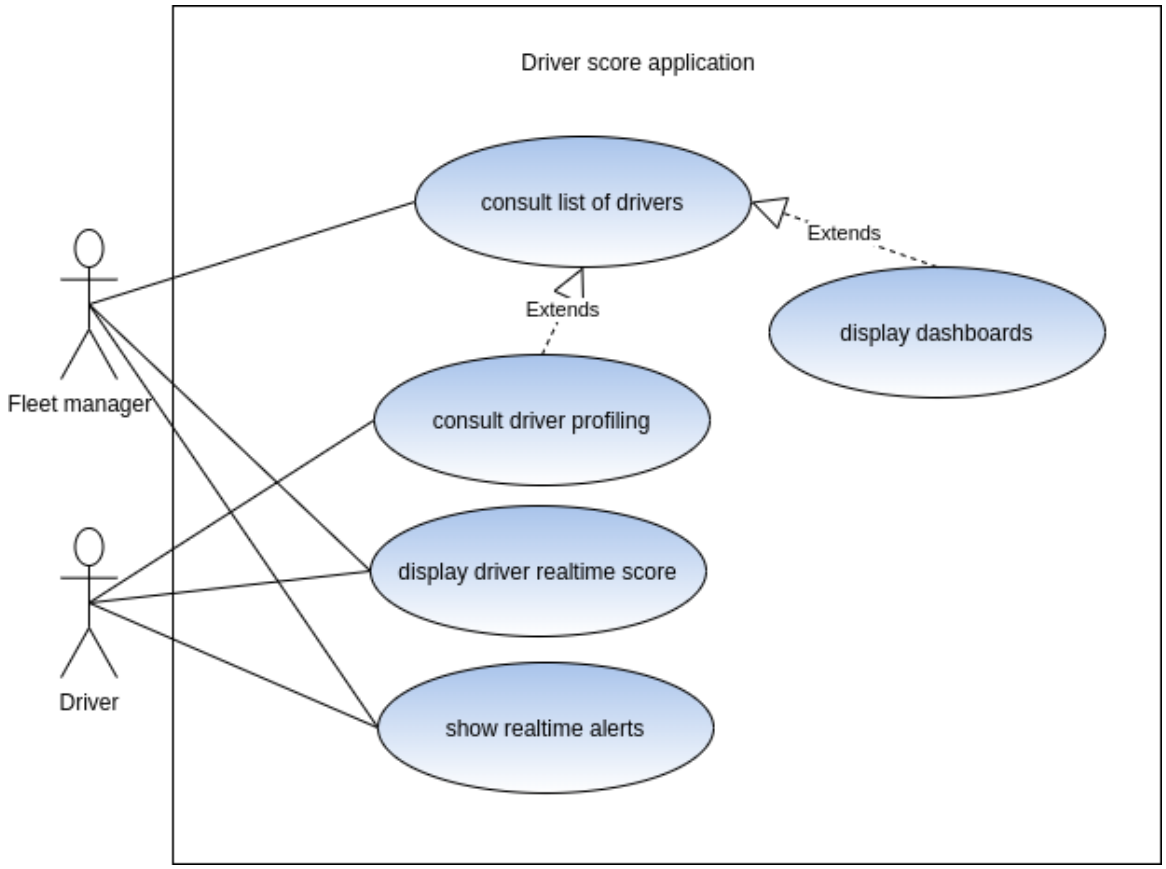


Figure 3.1: Use Case diagram

3.1.4.2 System sequence diagrams

The figure 3.2 and the figure 3.3 describe the interaction between our system and the concerned actors. After connecting to the application, a map on which the real-time score and alerts are indicated is displayed for both fleet manager and driver.
As illustrated the diagram 3.2, the fleet manager is able to access to a list of drivers and select the wanted driver and consult his profiling ; his personal information and his trips history are accessible. He is also able to evaluate all drivers by referring to the analysis made exposed via dashboards that allow him to figure out a good drivers from the risky ones.
For the driver, as described in the diagram 3.3, he is able to check his performance behind the wheel during the trip. He can consult his profile and be acquainted by his trips made and the way he was driving.

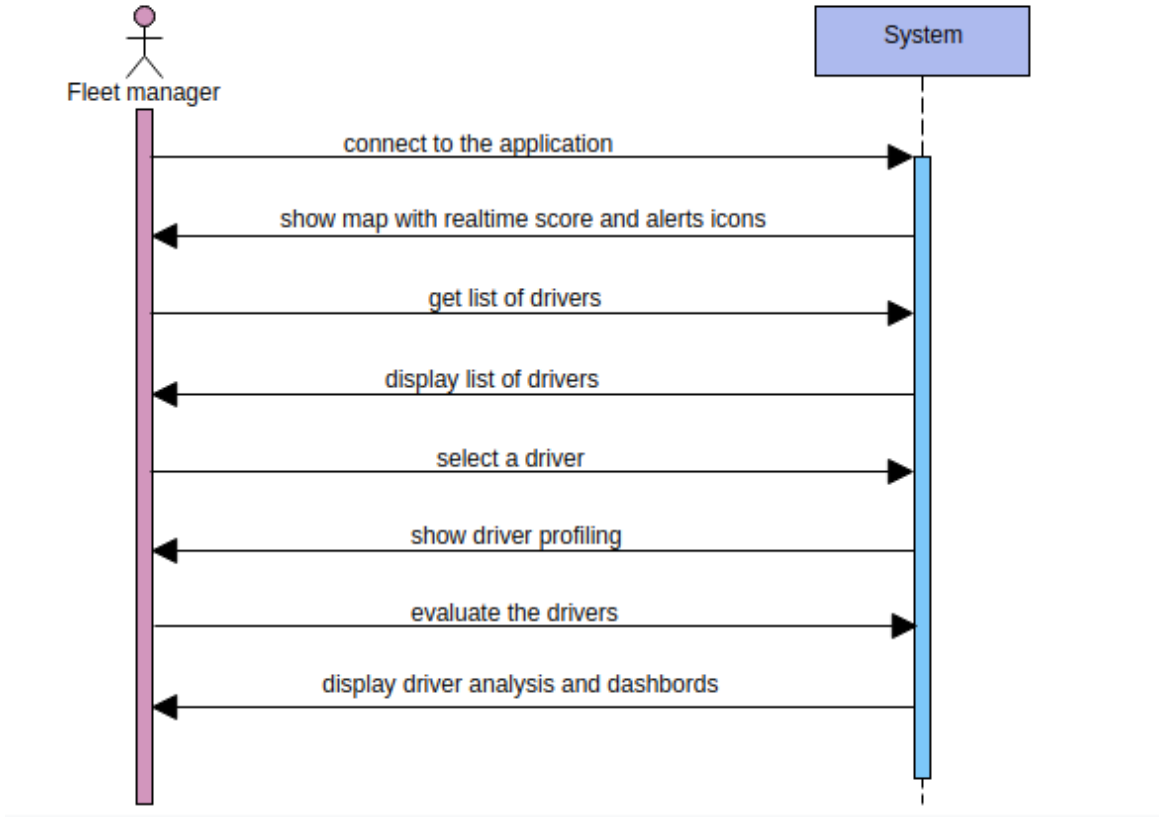


Figure 3.2: Display drivers profiling and dashboard

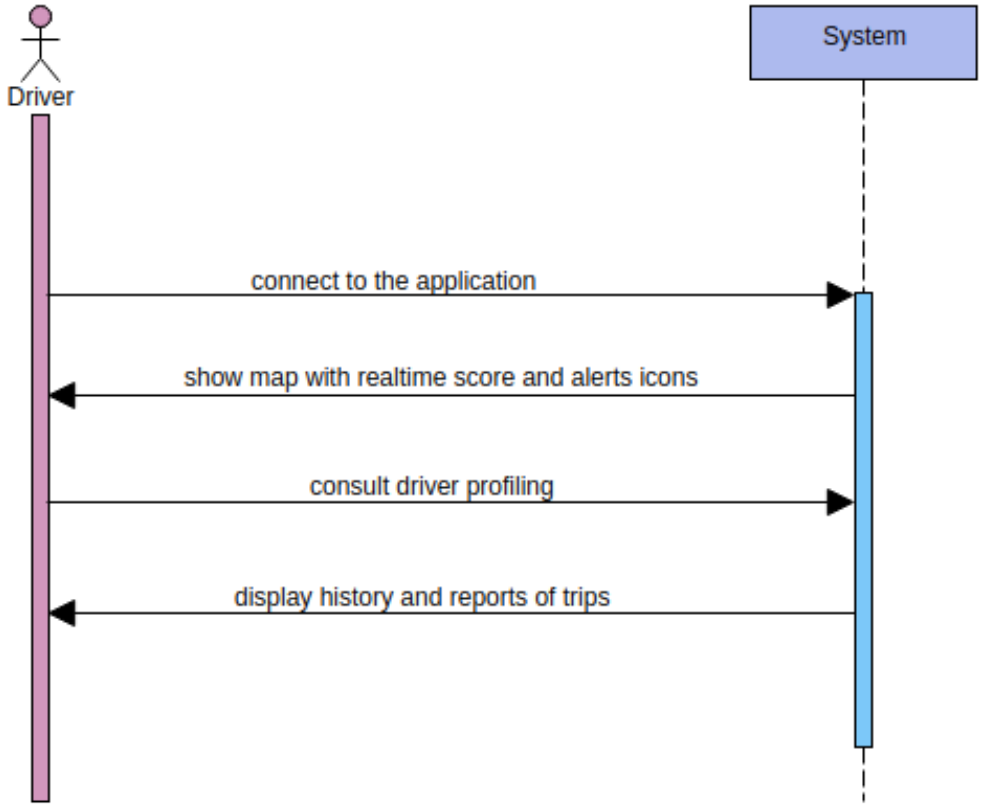


Figure 3.3: Consult driver profiling

3.2 Global Design

In this section we are interested in the overall design. For this purpose, we start by presenting the logical architecture then the physical architecture.

3.2.1 Logical architecture

Actually, there is a wide variety of tools and techniques that allow us to design and build a Big data solution that fits our needs. In the following, we present the most popular architectures that are Lambda and Kappa and we justify the choice we made.

3.2.1.1 Lambda architecture

Created by Nathan Marz and James Warren, lambda is defined as a generic and scalable architecture designed to to tackle complex issues combining massive quantity of data by taking advantage of both batch data processing and real time processing [26]. The concept of this architecture is to build a Big Data system composed of three layers. Each layer satisfies a set of properties and functionalities.

- **Batch Layer :**
  The Batch layer must be able to provide essentially two features:

  1. Storing immutable data whose volume increases continuously.
  2. Applying massive and regular processing on this stored data.

  The batch layer stores a master copy of all the data in a distributed way. This copy is immutable and is constantly increasing. The data is stored without any transformation made. Keeping the data in a raw format ensures that there is no data loss and this maintains a higher flexibility. This layer is unable to guarantee the low latency criteria. It is responsible for launching periodically processing on this data to precalculate the results in the form of a logical view. These views are called "batch views" [26].

- **Speed Layer :**
  the role of this component is to process new data streams in real time without any preprocessing to correct data sets and calculate incremental views that will complement batch views to provide more accurate results. This layer minimizes latency and provides real-time views with the most recent data. It is a continuous mode operation for each new data. The results, real-time views, provided by this layer are not as reliable as those of the batch layer [26].

- **Serving Layer :**
  this layer is in charge of storing and merging results of the batch and the real-time analysis from the speed and batch layers [26]. The serving layer acts as a database in which the stored information could be exposed to users and responds to ad hoc requests .

The figure 3.4 shows a presentation of the Lambda architecture as well as the interactions between the different layers.

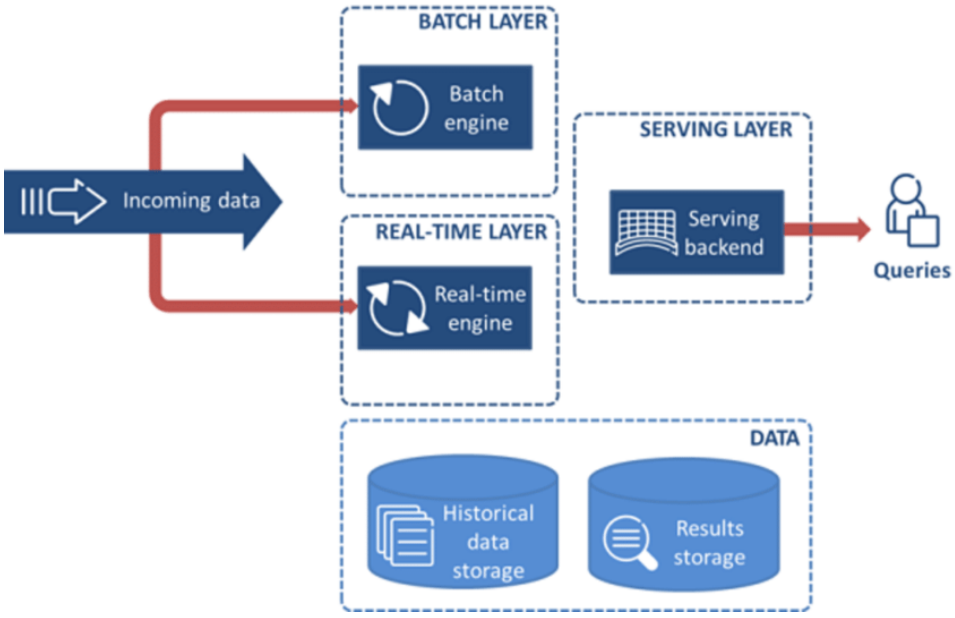


Figure 3.4: Lambda architecture [26]

Lambda architecture has several strengths since it allows to :

- Retain the raw data to reprocess in necessity;
- Offer a certain independence between architecture and technology;
- Tolerating faults thanks to the immutability of data and the repetition of processing;
- Present the most recent results to the client.

This does not negate the fact this architecture has no limits. It has the following disadvantages :

- Latency can be high;
- Each layer is based on several technologies which must be mastered by the developers;
- Business logic is implemented twice.

3.2.1.2 Kappa architecture

The Kappa architecture is developed by LinkedIn. The difference between Lambda and kappa is that Kappa has merged the speed and the batch layers to overcome the complexity of Lambda. So, it becomes composed of only two layers : the real-time layer and the serving layer as describes the figure 3.5.

The principle of Kappa is to handle batch processing and streaming using a single stream processing engine. This change also brings a modification on the DBMS which must be an immutable log file system. The streaming layer executes the processing in

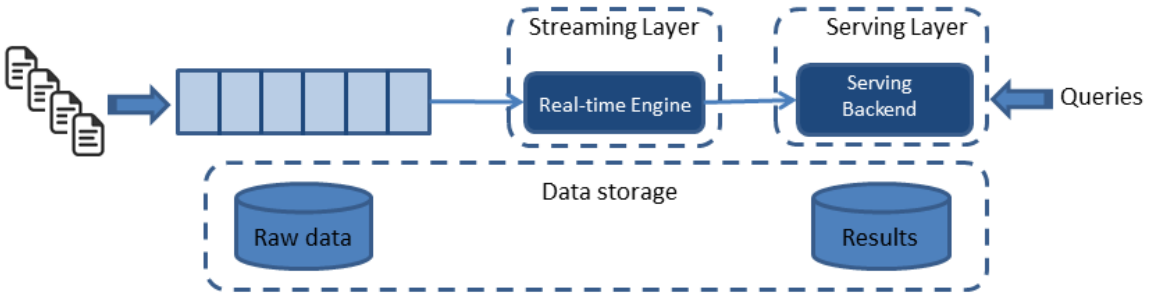


Figure 3.5: Kappa architecture [27]

real time. Normally, this type of job is performed for real-time data, but in Kappa, all data is considered recent.
Data reprocessing is only done when the code for this layer is changed. Reprocessing is carried out by launching another modified real-time processing and by duplicating the old results to replace them with the new ones at the end of the job. The serving layer is used to query the results as in Lambda architecture.
Kappa architecture has several advantages[27], we cite :

- Near real-time results;
- Technology independent architecture;
- One management system.

Kappa architecture also has drawbacks :

- It is not intended for data storage, but only for their processing;
- There is no separation between needs.

3.2.1.3 Adopted architecture

Choosing the ideal data architecture is not an easy task. As a result, we have to ask the right questions to be able to choose the appropriate architecture.
Given the previously described architectures characteristics and since our project is interested mainly by the real time driver behavior as low latency is defined as the most important criteria, the Kappa architecture is the most suitable architecture for our project.

3.2.2 Physical architecture

In order to guarantee good performance and reuse of our system, we opted to follow a 3-tier architecture.
The figure 3.6 shows the physical architecture of our application.

- **Presentation Layer :** corresponds to the user interface. Its role is to display the data and permit the end user to interact with it;

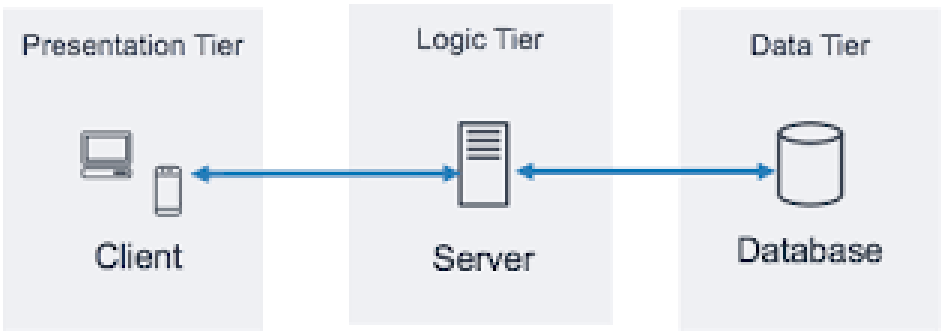


Figure 3.6: 3-tier architecture [28]

- **Logic tier :** It includes business logic and is responsible for providing the resources requested by the client;
- **Data tier :** It contains the DBMS and it allows centralization, storage and access to data.

3.2.3 Application flow process

The figure 3.7 describes the different data processing pipelines defined for of our application.

- **Data sources :** In each asset (vehicle, bus ..) a telematics device is grafted and delivers multiple information including speed, harsh braking, harsh acceleration and harsh corning;
- **Data ingest :** The reception and acquisition of large-scale data from installed devices for instantaneous use or storage in data lakes;
- **Data processing :** It is a series of processes and computational models that allow to return actionable results from gathered raw data;
- **Data storage :** Once processed, the output obtained will be hosted and preserved in storage systems;
- **BI and visualization :** It is a combination of graphical elements, accompanied by statistical analyzes to better interpret huge volumes of data.

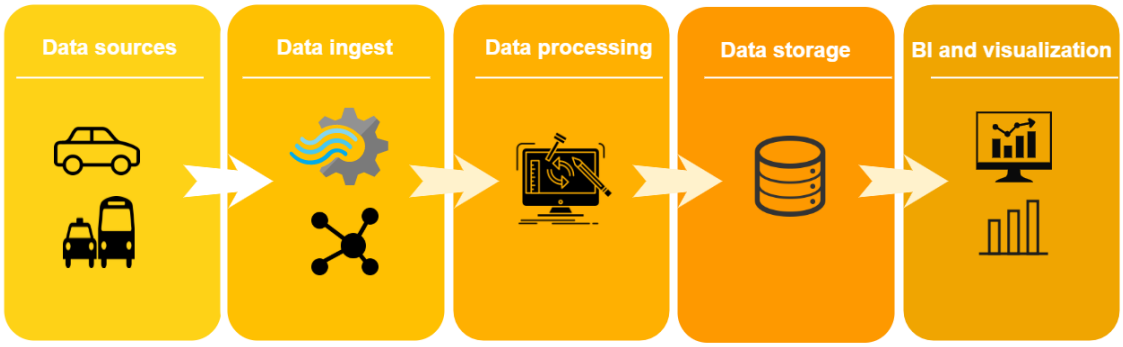


Figure 3.7: Dataflow design

3.3 Product Backlog and Sprints planning

In this section, we present our product backlog and how sprints are planned.

3.3.1 Product Backlog

To ensure the conformity of the final product to the customer's requirements, all the functionalities are brought together in the Product Backlog which represents the main artifact in each project.
The table 3.1 shows the Product Backlog.

ID-F	Features	ID-Us	User Stories
1	Data Cleaning	1.1	As a developer I want to receive data from a data lake source
		1.2	As a developer I want to examine and filter the validate records from the incorrect ones
2	Driver score service	2.1	As a developer I want to define the basis concepts and given factors for score algorithm.
		2.2	As a developer I want to define a computational model that calculates the score.
		2.3	As a user I want to be updated by the driving score in real-time.
		2.4	As a user I want to be updated by the harsh events occurred during the trip in real-time.
3	Data analysis	3.1	As a developer I want to store processed data in a storage system.
		3.2	As a developer I want to explore and analyze stored data .
		3.3	As a developer I want to create dashboards.
		3.4	As a user I want to visualize dashboards.
		3.4	As a user I want to visualize driver profiling.
4	Implementing a CI/CD pipeline	4.1	As a developer I want to create a pipeline and launch a build automatically following a code "commit".
		4.2	As a developer I want to deploy the code independently of the environment.

Table 3.1: Product Backlog

3.3.2 Sprints planning

One of the most important meetings of a Scrum project is planning sprints.
We organized to divide the project into four sprints as illustrated in the figure 3.8 :

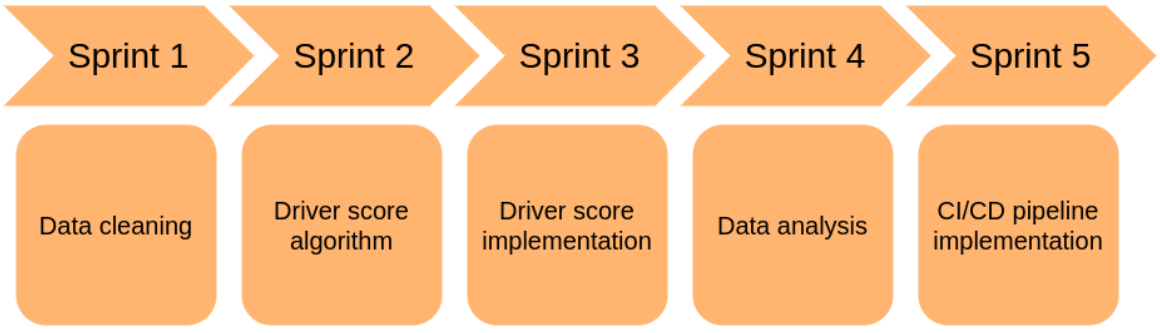


Figure 3.8: Sprints planning

As described in the figure 3.8, the first sprint will be dedicated to data pre-processing and cleaning. The second sprint will be about determining the algorithm in line with

our need then an implementation will take place in the third sprint. Finally, in the last sprint an implementation of a CI/CD pipeline for our application preceded by data analysis during the fourth sprint.

Conclusion

In this chapter we have identified the features that must be offered by the system. Then, we presented the architecture chosen for the realization of this project. Finally, we went around on the product backlog and planning of sprints. In the next chapter, we focus on the realization part.

Chapter 4

Achievement

Introduction

In this chapter, we present the implementation phase of the application and the realization of the various functionalities described in the previous chapter. We describe, in a first part, the hardware and software environment as well as the tools used. In a second part, we expose the attained work via screenshots by detailing each sprint of the project.

4.1 Hardware Environment

During the project development,we used the computer with the characteristics provided in table 4.1.

CPU	i5-7200U CPU @ 2.50GHz 2.71GHz
RAM	8 GB
Hard Drive	1 TB
Graphics processing unit (GPU)	Nvidia Geforce 920M
Operating system	Linux - ubuntu 18.04

Table 4.1: Characteristic of the used computer.

4.2 Software Environment

Several tools are connected together to meet our needs. In this part, we spot the light on the chosen technologies used to implement our solution.

4.2.1 Big Data tools

4.2.1.1 Apache Flink

Choosing the right technologies is a challenge for the company in order to satisfy their needs and to be convenient for their business requirements. As stream processing

frameworks differ in their capabilities by virtue of characteristics and parameters, a comparison summarised in the table 4.2, is made to determine the convenable tool.

	Storm	Spark	Flink
Processing model	event at time	micro-batch	event at time
Memory management	Hadoop	Spark	Flink
windowing	time based	time based	flexible
Latency	low	medium (seconds)	high (sub-seconds)
Fault tolerance	record ACKs	RDD based	checkpointing
Delievery guarantee	at least once support	exactly once	exactly once
Throughput	high	medium	high
Advanced features	not support	support	support
APIs	Complex	simple	simple
Community	Selective	Wide	Growing
Language	Java,Clojure	Java, Scala, python	Java, Scala, python

Table 4.2: Comparison of stream processing frameworks [6]

As a conclusion, by dint of its rich features as well as its widely hit, we choose to work with flink as it represents an effective solution for our case.

4.2.1.2 Kafka

Kafka is a high-performance, real-time messaging system. It's an open source tool and become a part of Apache projects in October, 2012 [29].
Kafka is a distributed and partitioned messaging system that highly fault tolerant and scalable. It has been tested to process and send millions of messages per second to several receivers. It includes five basic APIs [29] namely Producer API, Consumer API, Connect API, Streams API and AdminClient API.
Among the use cases of Apache kafka, it can be used to process a continuous stream of information in real-time and pass it to stream processing systems such as Flink as illustrated in the figure 4.1.

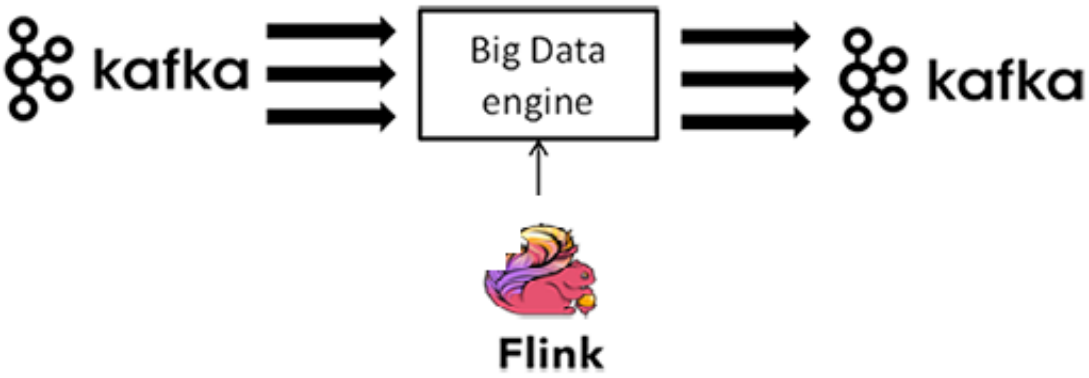


Figure 4.1: Kafka with flink

4.2.2 Management and Development tools

4.2.2.1 Bitbucket

Belonging to the Atlassian family, Bitbucket is a code repository management solution, compatible with the version management system GIT and Mercurial [30].
Designed from the start to serve professional developers, Bitbucket allows development teams to collaborate, update, review, control, test, validate and deploy their source code, in complete security. Comparable to the very popular GITHUB, it nevertheless has advantages making it in some cases more powerful.

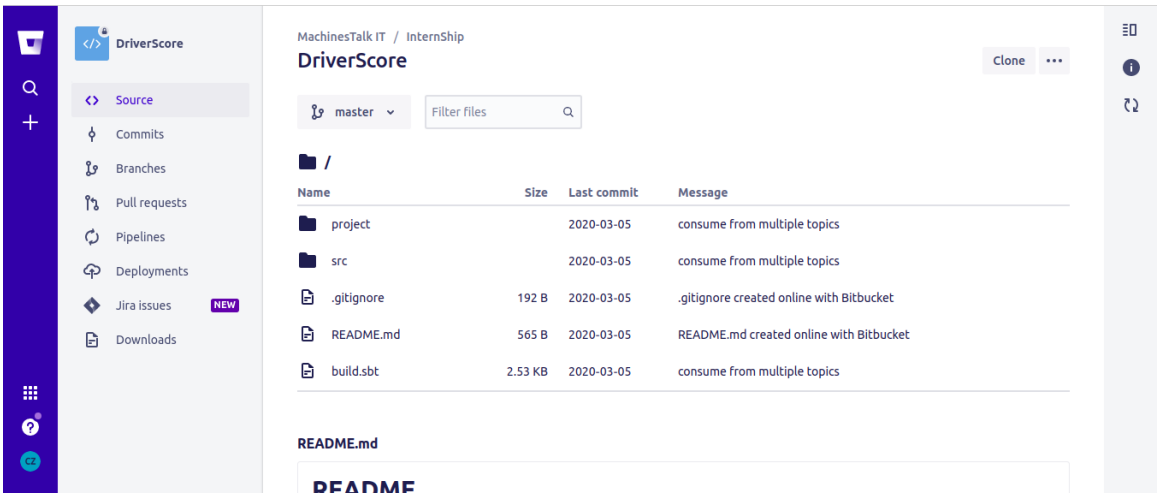


Figure 4.2: Bitbucket interface

4.2.2.2 Git

Git is a decentralized version management software, created by Linus Torvalds, author of the Linux kernel, and distributed under the terms of the GNU General Public License version 2 [31]. This tool allows to have an history on all modifications made

on a project. Added to that, it handles the problem of conflict by merger.
Git is distinguished by its speed and its management of the branches which allow the development of new functionalities in parallel.

4.2.2.3 Jira

Developed by Atlassian, jira is a software mainly dedicated to the management of development projects. It is a tool that is ideal for project managers who adopt the agile approach. Thanks to its functionalities, this it allows to schedule tasks with great flexibility by obtaining precise estimates or even prioritizing user stories according to their importance, to track the workflow and to release excellent software.

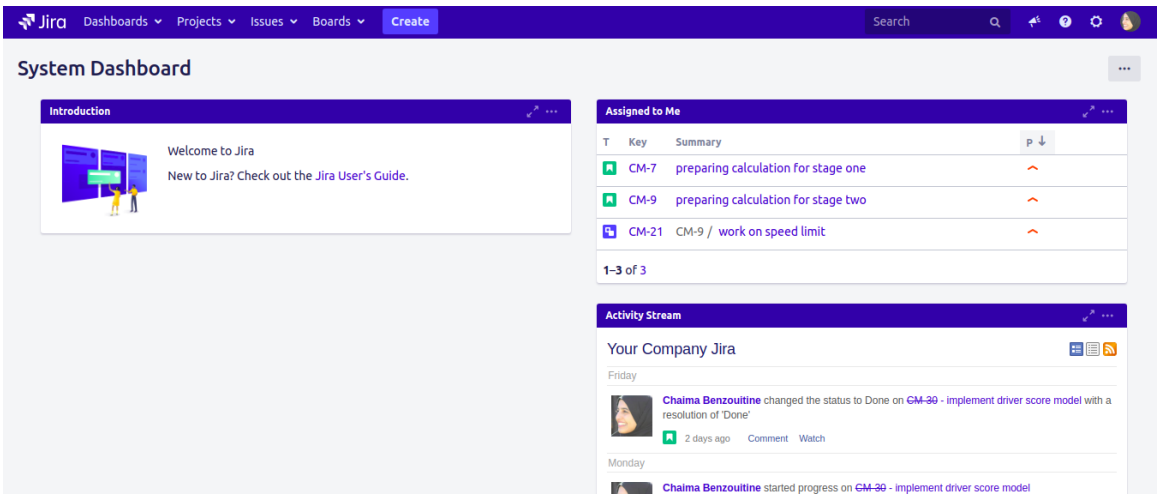


Figure 4.3: Jira interface

4.2.3 Data analysis tool

For data analysis, we adopted Metabase which is an open source business intelligence and data visualization tool with SQL functionalities. Unlike other data analytics tools which require free use conditions, Metabase is available as a free version. The quality of its interface and its simplicity of handling also make it a powerful and the easiest tool, as well as the richness of its functional coverage such as analysis, dashboards and shared results with company members. It has a variety of graphical representations and sustains different backend databases for instance mysql, mongoDB and Postgres [34].

4.2.4 Programming language

Scala : is a recent static-typed language that compiles to Java bytecode and runs on the JVM. Its name is the concatenation of Scalable Language. It was designed at the federal polytechnic university of Lausanne (EPFL) to be a versatile, powerful language suitable for small scripts as well as large software. Martin Odersky, its creator, is a programming professor at EPFL. He has long sought to unify Object-oriented programming (OOP) and functional programming (FP). On the one hand, Scala is a pure object

language. Classes, constructors and inheritance will all be there, but with a syntax much less verbose than that of Java. On the other hand, Scala supports the functional paradigm, functions are objects just like strings and integers so they can be passed as arguments to other methods or stored in variables [35].

4.2.5 DevOps tools

4.2.5.1 Docker

Docker is a tool which is used to automate the deployment of applications in lightweight containers so that application can work efficiently in different environments.
Docker provides the suitable frameworks for each different applications. With Docker technology, containers will be treated as modular and light virtual machines. These containers offer great flexibility since they can be created, deployed, copied and moved from one environment to another, which makes it possible to optimize applications [38].

4.2.5.2 Jenkins

Written in Java, Jenkins is defined as a continuous integration tool. It is an open source tool that allows to test and report modifications made on a wide code base in real time. By using this software, developers can detect and resolve problems in a code quickly. So testing new builds can be automated, making it easier to incorporate changes to a project on a continuous basis. Jenkin's goal is to accelerate software development through automation [39].

4.3 Features Release

Our work is divided into sprints according to the scrum methodology. In this section, we detail each sprint during the development cycle of our application that started with cleaning the data to prepare it to the processing stage. Before detailing the implementation part in the third sprint, we determine the algorithm chosen to calculate the score. The firth sprint takes into consideration the analysis of data and we ends with the implementation of CI/CD pipeline to deploy the obtained application.

4.3.1 Sprint 1 : Data Pre-processing

The volumes of data we have are constantly increasing, as are the risks of error. The most common causes of data inaccuracy are missing and incoherent values that can be happen due to several circumstances for example unreachable network zones, damaged sensor devices and synchronization issues. Therefore, we must rely on data cleaning to optimize our data management processes and strengthen the integrity and relevance of our data by reducing inconsistencies, avoiding errors before proceeding with any type analysis.
The realization was in the form of a code written in scala. This implemented model defines certain standards for verifying the validity of incoming data.
The technique applied to deal with data problems while pre-processing stage can be summarized in the figure 4.4

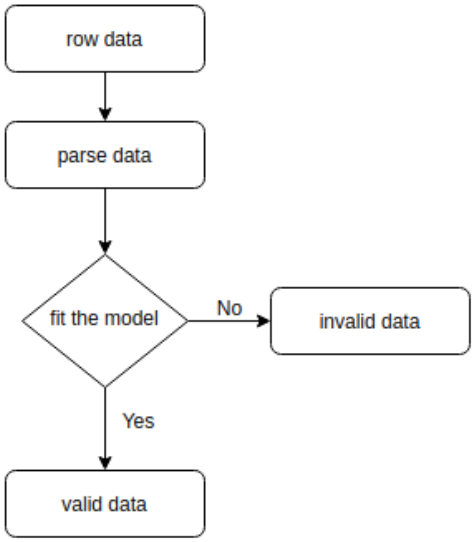


Figure 4.4: Pre-processing flow

The entry data is parsed to a specific model defined by a set of rules that detect if the message having the entire fields correctly represented and there are no missing values. Two cases are depicted :

1. If the row data fits the model then it will be marked as valid and will be sinked to the appropriate topic to be processed. In the figure 4.5, the received messaged is correct and saved in the data clean topic.

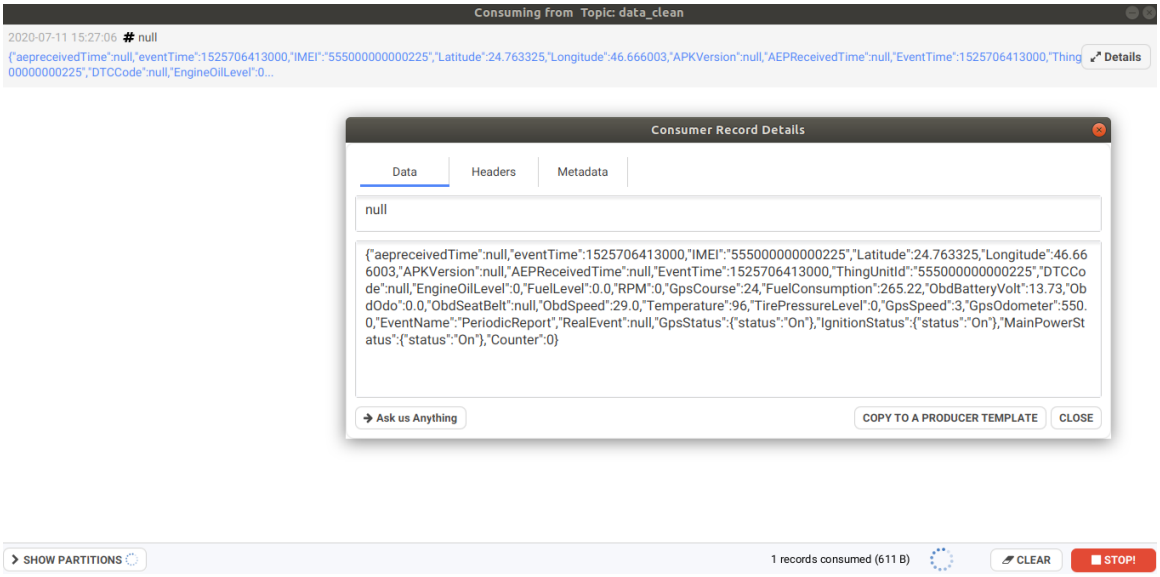


Figure 4.5: Data clean input

2. If the row data don't fit the model then it will be dropped to a topic namely device-log-device as shown in the figure 4.6 with the corresponding error mes-

sage to be analyzed later. Through this analysis, we can discover the main reason behind the wrong data imputation.

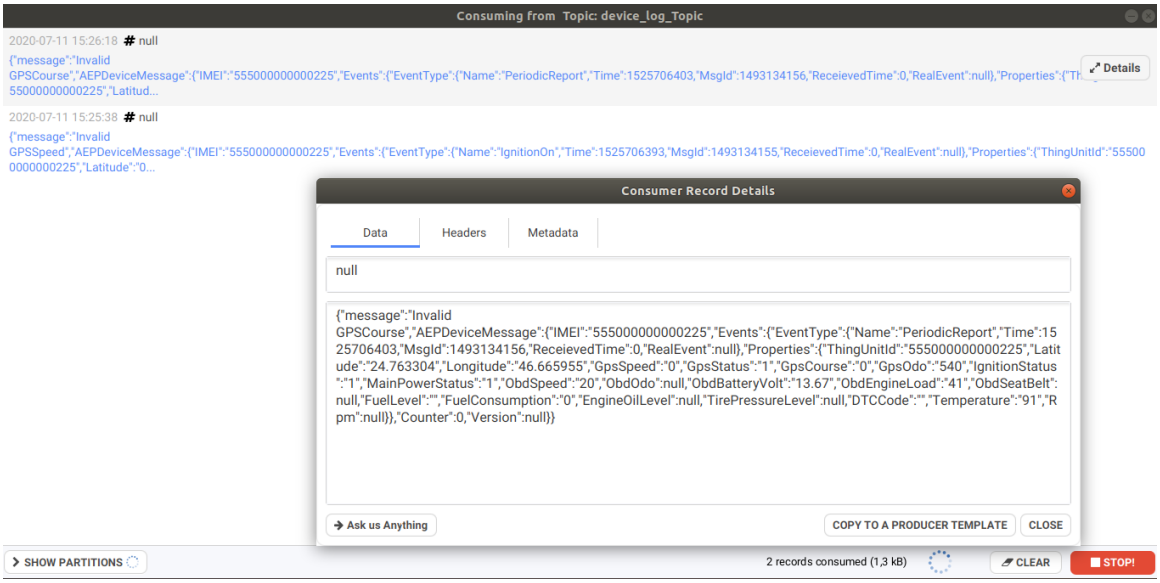


Figure 4.6: Device log topic output

4.3.2 Sprint 2 : Driver Score Algorithm

The driving Score model provides broad and measurable benefits for fleet managers and drivers. Valuable insights into the manner how people drive can be extracted from analyzing data gathered by sensors. In order to achieve that, the approach of determining the score is based on evaluating the changing state of the vehicle momentum, so that a flow of continuous data is collected and assessed. The following work was determined in cooperation with Machinestalk business team and approved by Product and Business Development department.
The driver score is established referring to the following factors:

1. Violation type
2. Number of violations made
3. Distance driven

The work is divided into two stages as shown in the figure 4.7. The first stage will be dedicated to the accumulation of the penalty points made. The second phase determines the score in percentage based on the distance driven.

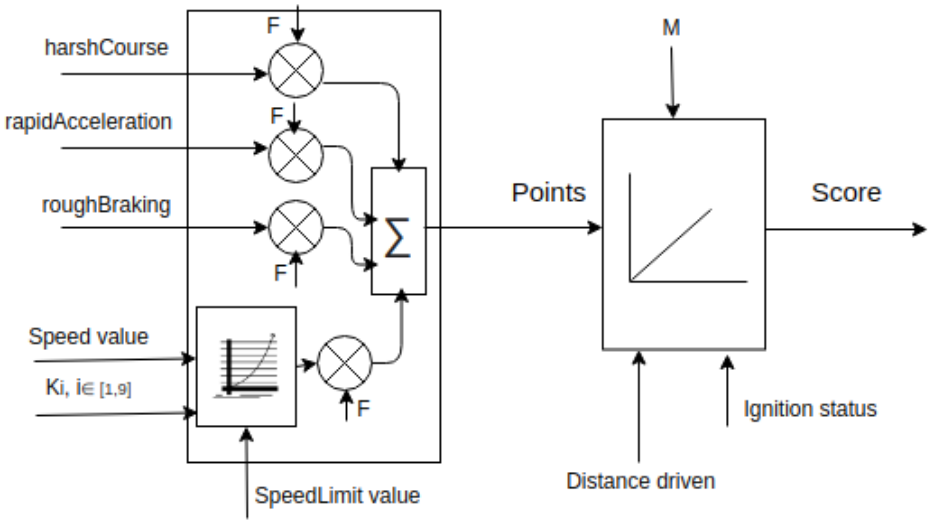


Figure 4.7: Work structure

4.3.2.1 Stage one

- **Speed violation**

  Each speed value should be compared to the speed limit plus the error allowed. When the speed value is greater than the limit and lay down in one of the nine regions as shown in the figure 4.8, a counter variable should be incremented by 1 for that region.
  While the driver repeats the same mistake more than one, the penalty will raise following the Fibonacci sequence.

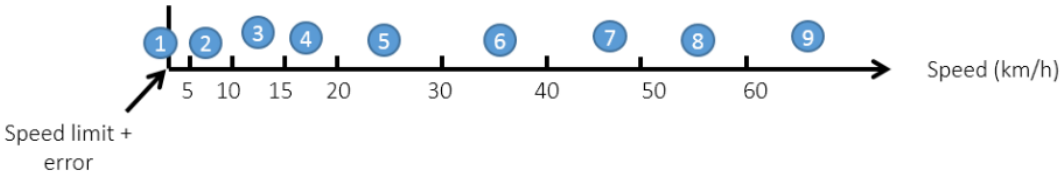


Figure 4.8: Speed regions

Obviously, when the speed value is high, the points lost will be greater than the points lost for low speed value for the package of data.
To calculate the points lost for this stage, we will divide the penalty points made by the k-value of each region which are the corresponding weight of the region summarized in the table 4.3.

To achieve this, the equation (4.1) will be applied in the current region.

$$lost_points_SV = \frac{Penalty_points}{K} \tag{4.1}$$

k1	k2	k3	k4	k5	k6	k7	k8	k9
20	17	15	10	7	5	2	1.5	1

Table 4.3: K-values parameters

- **Harsh driving behavior events**
  To each harsh event such as rapid acceleration, harsh braking a weight has been associated. The same concept of the speed violation has been effectuated; the number of violations made will be proportional to the Fibonacci series, that's mean that for example the weight of the fourth mistake will be 5 hence we get the lost_points_HE.

- **Total lost points**
  The total lost points will be the sum of the two previous metrics (4.2) :

$$lost_points = lost_points_SV + lost_points_HE \tag{4.2}$$

4.3.2.2 Stage Two

In order to describe the relation between the driver score and the points lost, we started with the idea that the more lost points are earned the less score is obtained. Thus the following equation (4.3) had been chosen :

$$Score = \frac{1}{lost_points} \tag{4.3}$$

The figure 4.9 shows that the lost points and the score are inversely proportional.

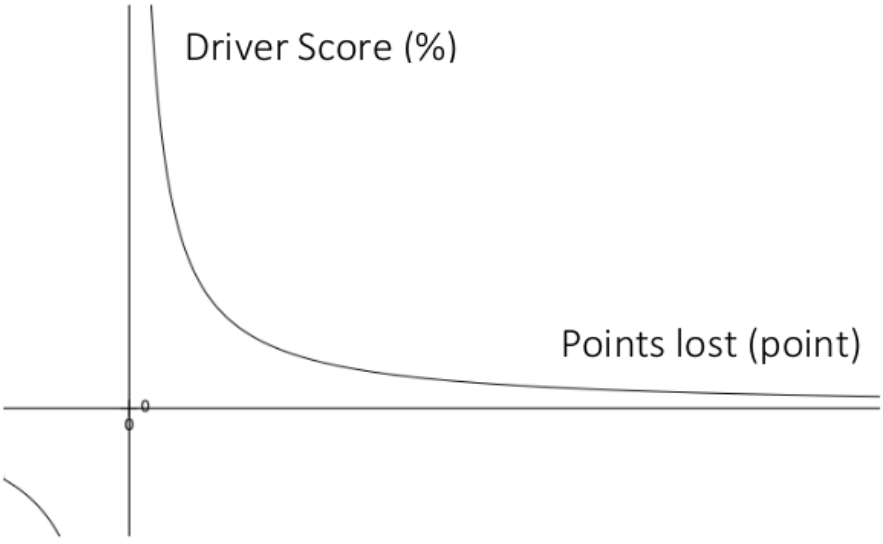


Figure 4.9: Equation (4.3)

Since the score is bounded between 0 and 100, the function should be calibrated and shifted to the left by k as describe the equation (4.4) :

$$Score = \frac{1}{lost_points + X} \tag{4.4}$$

In case of there are no violations made, the points lost should be 0 and the score should be 100.

$$Score = 100 = \frac{1}{X} \Rightarrow X = \frac{1}{100} \tag{4.5}$$

In consequence, we obtained the equation (4.6) :

$$Score = \frac{1}{lost_points + \frac{1}{100}} \tag{4.6}$$

However, the decaying rate of equation (4.6) is very fast. To decrease the rate, the equation should become :

$$Score = \frac{C}{lost_points + \frac{C}{100}} \tag{4.7}$$

By analysing the equation (4.7), we observe that the higher the C value, the lower the decaying rate gets and as we want to ensure that scores stay proportional and fair regardless of distance driven. As result, the C value is substituted by the distance driven to get the equation (4.8)

$$Score = \frac{distance_driven}{lost_points + \frac{distance_driven}{100}} \tag{4.8}$$

Finally, to avoid the biased model that could be occur when the distance is too short, we multiplied the equation (4.8) by the M factor which was set at 100 points per kilometer and we reached the final equation (4.9) :

$$Score = \frac{distance_driven * M}{lost_points + \frac{distance_driven * M}{100}} \tag{4.9}$$

The figure describes the aspect of the equation (4.9).

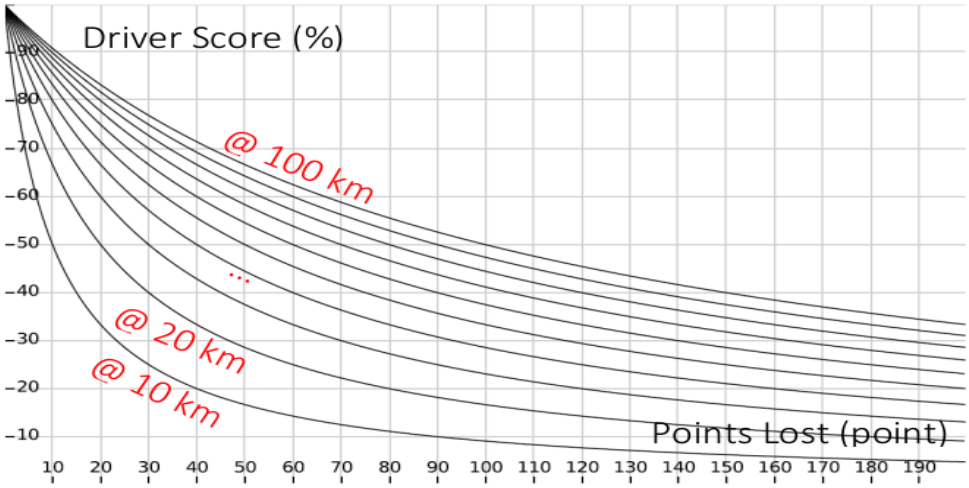


Figure 4.10: Equation (4.9)

4.3.3 Sprint 3 : Driver score implementation

The implementation of the driver score algorithm was done in several tasks. At this stage, we present and detail the different tasks we have gone through. The figure 4.11 gives a global view of the followed process for the implementation starting with ingesting the data to the storage phase.



Figure 4.11: Driver score implementation

4.3.3.1 Speed limit determination

In order to determine the corresponded speed limit value in a given zone, we have recourse to the google Roads API service. It maps GPS coordinates to road geometry and specifies the speed limit all over these road segments. The availability of this API is through clearly a HTTPs interface which reveal various services such as Speed limit [36].
The Speed limit service returns the value of the maximum speed in such given segment of road. Since the data provided by this service is not in real-time and in order to minimize the number of requests which is costly, we gathered the speed limit values as well as the corresponding latitude/longitude pairs in a data-set. As a consequence, the application gets instantly the speed limit parameter from the database.

4.3.3.2 Apache flink windows mechanism selection

Flink windows represent the core of processing unlimited streams, they aim to split the stream of incoming data into a finite size packs over which computations could be applied. The creation of a window start with the arrival of the first element and it fades right away the time exceeds its end [37].
Defining the way how streams are divided and assigned to windows is commanded by the window assigner. To choose the suitable window assigner for our use case, we have first studied the most important examples which are global windows, tumbling windows, sliding windows and session window.

- Global windows : elements having the same key are allocated to a single global window.
- Tumbling windows : elements are assigned to a specified size and non overlapping window.
- Sliding windows : elements are assigned to a fixed size window that can be overlapped, hence, elements can belong to more then one window
- Session windows : elements are grouped by sessions of activity that are bounded between a particular point of time and a given time if inactivity.

To fit our application needs, we choose the global window as a way to handle streams. A window is assigned to all trips made by a driver with a specific vehicle (device imei) as described in the figure 4.12. Trips are composed of each trip that starts when the vehicle is turned on (ignition status parameter is on) and ends as soon as the ignition status parameter is turned off.

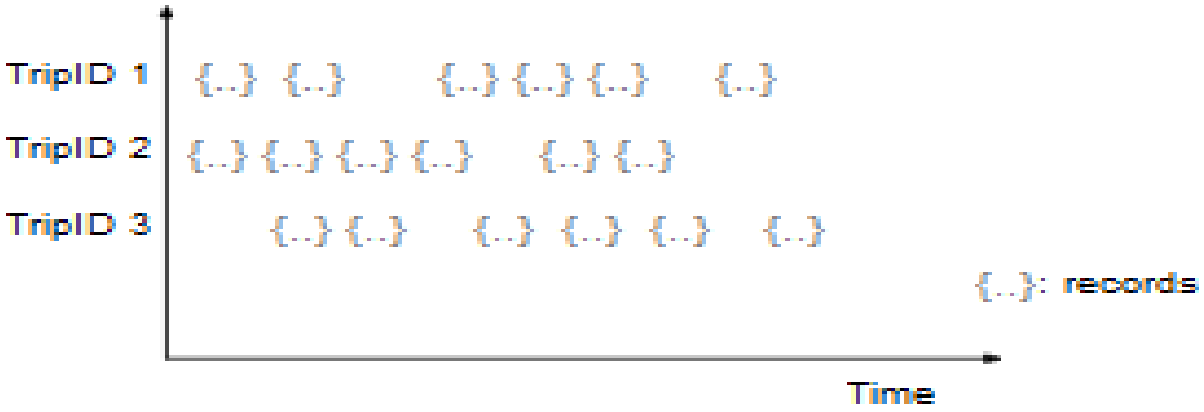


Figure 4.12: Global Windows

4.3.3.3 Ingestion and storage of data

To handle this task, we used Kafka Connect, an open source module of Apache Kafka, which is a tool that enables Kafka to connect with external data systems such as databases by making the data disposable for stream processing with the low latency feature [32]. It models the simple definition of connectors having large data traffic in and out of Kafka as illustrated the figure 4.13.
With the aim of obtaining the speed limit parameter from the database, a kafka source connect is implemented to make it available and stored in the specific topic to be ingested. And for retaining computational data in a storage system, we used kafka sink connect to stock data from topic to the database.

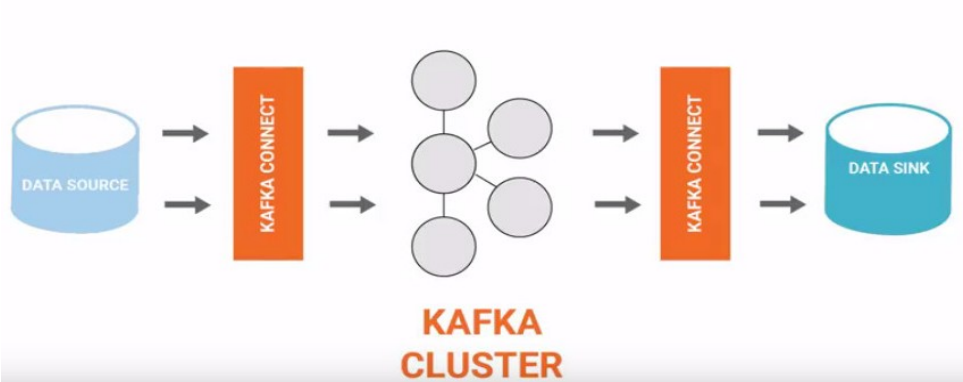


Figure 4.13: Kafka Connect [32]

4.3.3.4 Screen shots

In the figure 4.14, we find the realtime score and harsh events made displayed in kafka sink, output topic, as a result from the implementation phase.

The figure 4.15 illustrates the storage of the generated computations in the MySQL database.

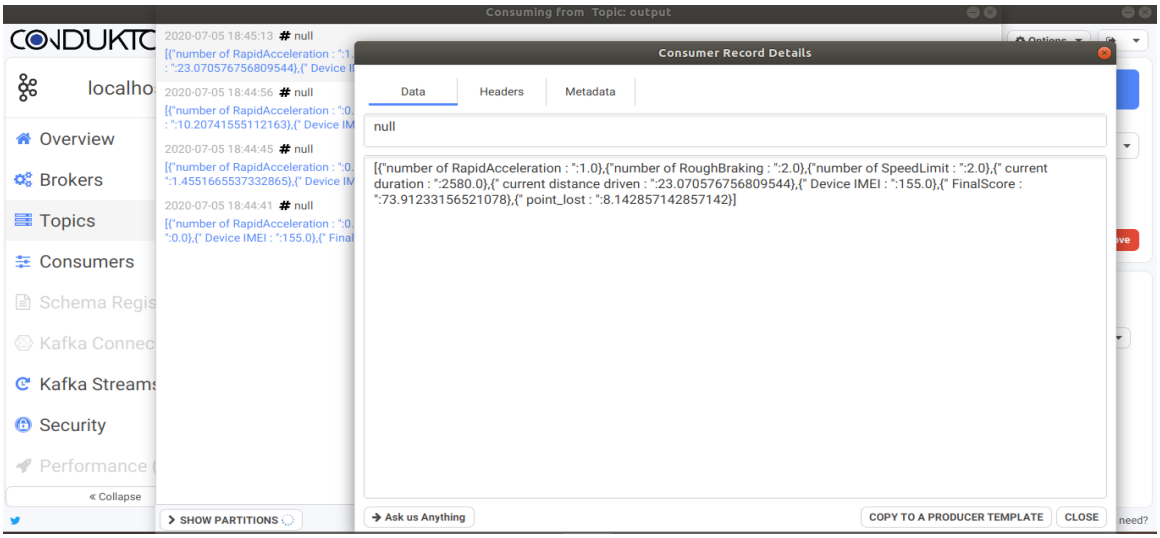


Figure 4.14: Kafka output

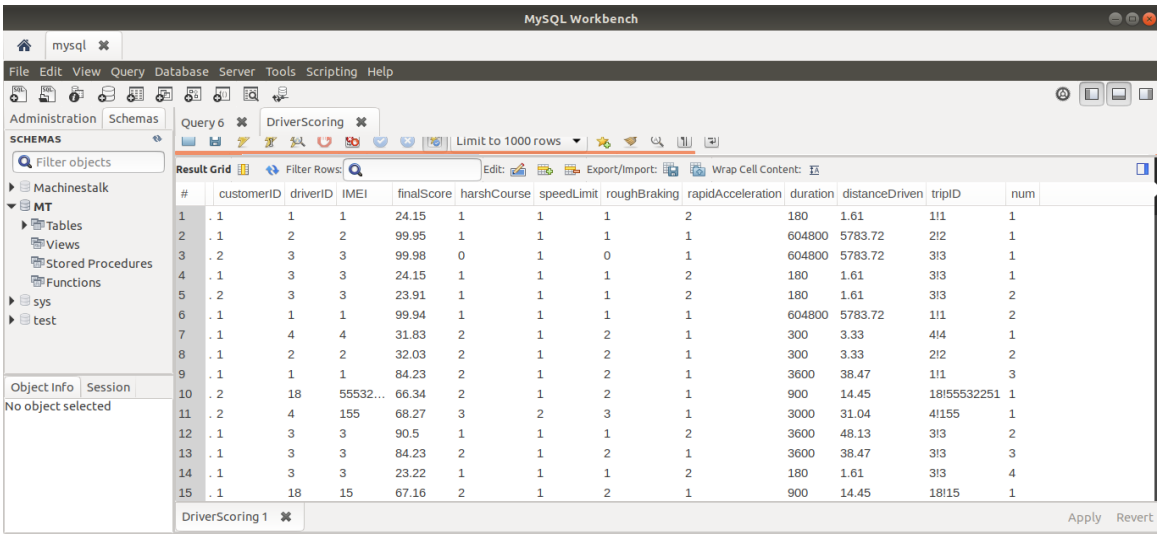


Figure 4.15: MySQL Database

4.3.4 Sprint 4: Data analysis

In order to provide the user with a more complete and exhaustive vision of a series of data, So metabase is this sort of business intelligence tool and a way of digging into generated data to facilitate its interpretation with a quite intuitive query interface.

The idea was to make the application beneficiers, especially the fleet manager, able to take decisions thanks to a representation that clearly prioritizes the issues. He can easily review his drivers scorecard and draws conclusions.

The first step after installing metabase platform was to connect our database to this tool via mysql driver and setting up the basic configuration. The exploration of the data will be through asking questions. A question button provides an editor zone to write down SQL queries.
For that, we prepared a set of actionable scenarios :

- Selecting top five drivers for a specific customer (figure 4.17):



Figure 4.16: Top 5 drivers

- Selecting the score worst driver for a specific customer (figure 4.17):

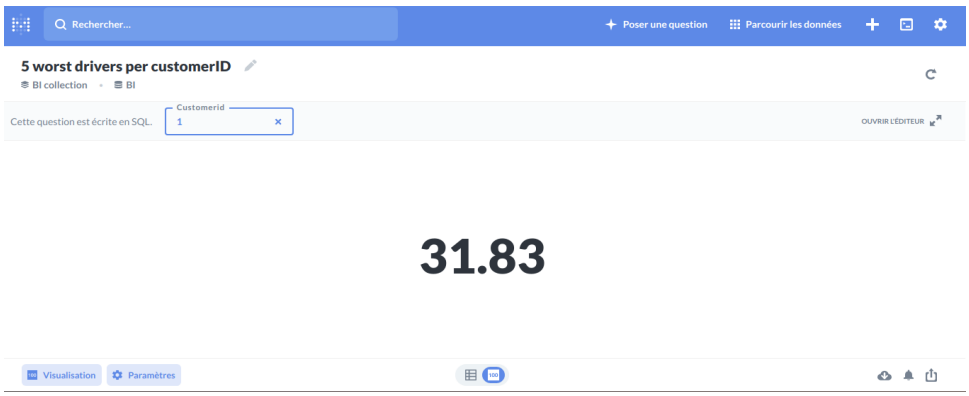


Figure 4.17: The score of the worst driver

- Selecting the total score over all trips madeby a specific driver for a specific customer (figure 4.18):

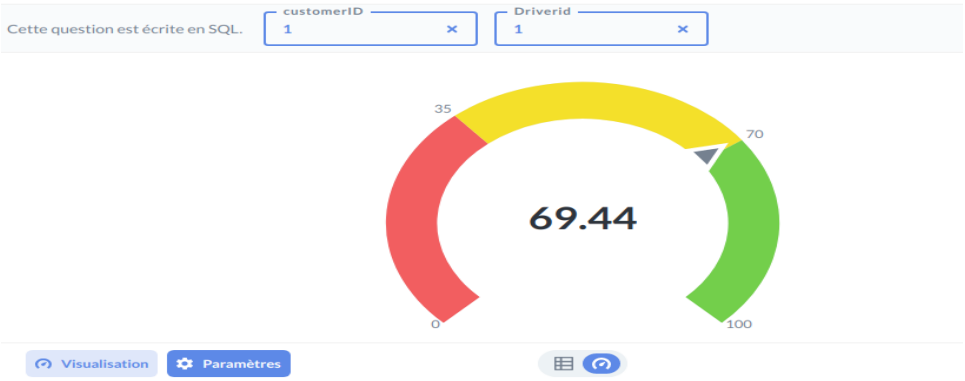


Figure 4.18: Total score

- Selecting the number of harsh events made by each driver for a specific customer from the higher to lower : (figure 4.19):

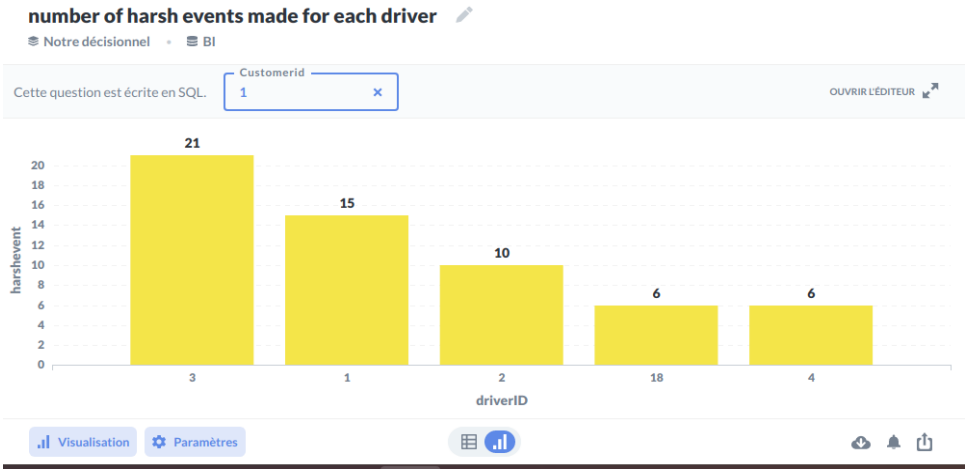


Figure 4.19: Number of harsh events for each driver

4.3.5 Sprint 5: Application deployment

To manage the lifecycle of our application before its deployment, we have implemented a continuous integration/continuous deployment pipeline.
The figure 4.20 describes all stages gone through in this part starting from the dev team to the deployment of the application.

The chain begins with a push from the developer to Bitbucket repository using the versioning tool management Git. When a new code is detected, our pipeline starts automatically : Jenkins will play the role of the conductor between the components of our platform and it will launch the stages, defined in the Jenkins file, one after the other until it reaches a new deployed version.

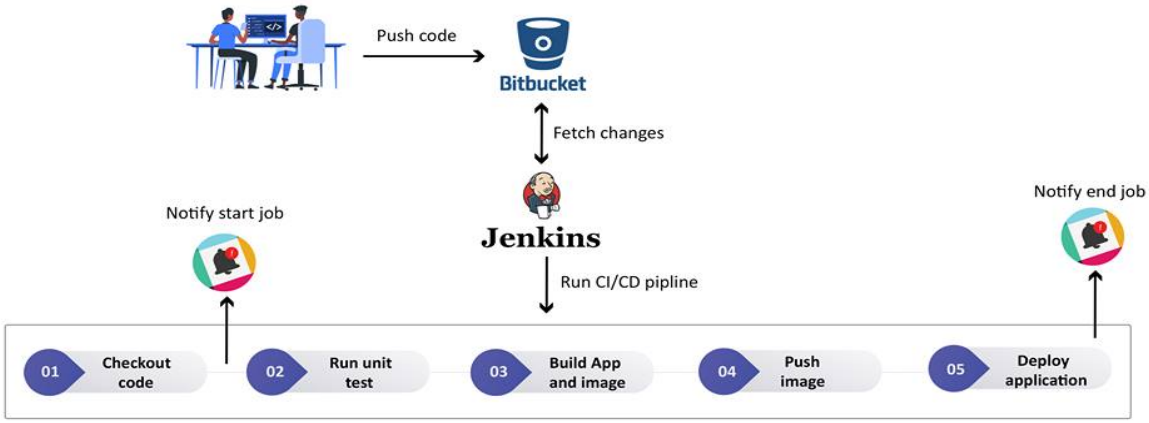


Figure 4.20: Application CI/CD pipeline

The stages are as follows :

1. **Checkout code :** Jenkins clones the new version of code from Bitbucket;
2. **Run unit test :** Jenkins runs defined unit test;
3. **Build application and image :** In this stage, Jenkins launches the build of the application. The output of this stage is an artifact (.jar file). Then, Jenkins creates docker image through a file namely Dockerfile;
4. **Push image :** Jenkins pushes the docker image in the private Docker registry which is a docker repository owned by the company;
5. **Deploy application :** Jenkins deploys a new version of the application in a docker container thanks to docker swarm, the container management engine that allows users to guide container deployment and automate updates, integrity monitoring and failover procedures.

The figure 4.21 determines the state of the pipeline stages on the Jenkins dashboard.

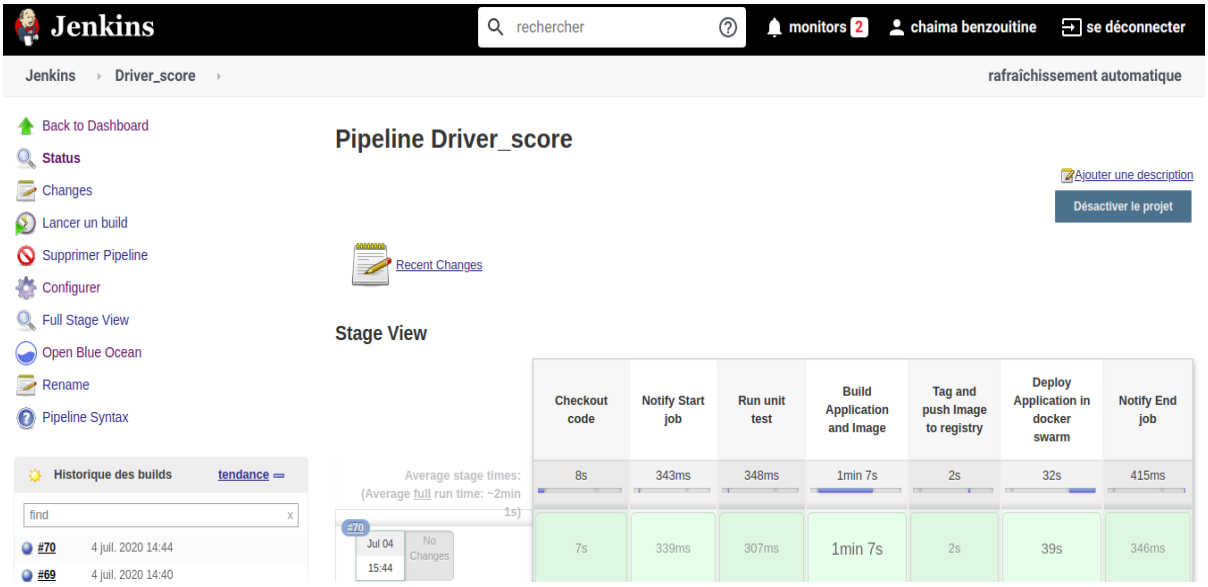


Figure 4.21: Driver score pipeline

Conclusion

With this chapter, we close the realization part of our application. The work achieved by each sprint was explicated and demonstrated using screen shots of each product released. This was preceded by a presentation of hardware and software environments used during the project.

General Conclusion

At the end of this report, we present what we have accomplished during the period reserved for this end of studies internship entitled "Driver score application".

This report details the stages we have gone through, the big data and deployment technologies that we used to implement a solution to survey and monitor driver behavior through a real time scoring.

We began first with presenting the general context by shedding the light on the company where our internship was held, defining the project context and the adopted methodology of work.

Then, we present a preliminary study of the basics of Big Data and stream processing concepts, followed by a review of existing solutions. This study allowed us to set the needs of the project which is monitoring driver behavior behind the wheel in real time in order to determine risky drivers from safe ones.

The realization of the project was done through several stages. First, we have retrieved the necessary data from the company's devices. Understanding the data is an extremely important and delicate task, that's why we have cleaned and prepared this data for the process phase in which the driver score model was implemented.

Determining the score formula was an interesting and challenging task. Diving in the mathematical domain was an exciting experience to determine the suitable formula for our company.

Thereafter, to exploit the generated data, we used a business intelligence tool to visualize the results and deduct valuable insights. And finally, to deliver a complete version of solution, we adopted the devops approach to manage the development cycle of our application.

The development of this project has allowed us to enrich our theoretical and practical knowledge. Theoretically, we were able to get started in the big data field. On a practical level, we have enriched our skills in terms of Scala programming. We were also able to learn the flink technology which is a new technology currently expanding in the field of big data. Beside that, it gives us the opportunity to discover the devops job and familiarize with its tools.

Regardless of the technical constraints and challenges faced, we achieved the objectives and we met the requirements of the project. As an enhancement, we intend to improve and exploit the achieved work by developing a complementary module that aims to predict the behavior of driver during a trip to reduce road risks.

Netography

[1] **Top 5 Use Cases of IoT in Transportation**, 10/03/2020:
https://dzone.com/articles/top-5-applications-of-iot-in-transportation

[2] **Machinestalk**, *About Machinestalk*, 10/03/2020:
https://www.machinestalk.com/

[3] **How to Implement DevOps with CAMS"**, 10/03/2020:
https://squadex.com/insights/how-to-implement-devops-with-cams/

[4] **Big Data Career Guide**, 10/03/2020:
https://www.simplilearn.com/big-data-career-guide-pdf?utm_campaign=What-is-Big-Data-What-is-Hadoop-CKLzDWMsQGM&utm_medium=Tutorials&utm_source=youtube

[5] **Big Data Can Do What? 9 Capabilities that Might Surprise You"**, 10/03/2020:
https://www.rasmussen.edu/degrees/technology/blog/big-data-capabilities/

[6] **Spark Streaming vs Flink vs Storm vs Kafka Streams vs Samza : Choose Your Stream Processing Framework**, 10/03/2020:
https://medium.com/@chandanbaranwal/spark-streaming-vs-flink-vs-storm-vs

[7] **End to end agility**, 15/05/2020 :
https://www.capgemini.com/2019/04/how-to-source-your-devops-teams/

[8] **Agile**, 25/03/2020:
https://kissflow.com/project-management/values-and-principles-of-agile-manifesto/

[9] **Scrum**, *Scrum process* , 25/03/2020:
https://www.apeironsoftware.com/scrum-roles-artifacts-and-ceremonies/

[10] **Devops automation**, 15/05/2020 https://www.redhat.com/fr/topics/devops/what-is-ci-cd

[11] **Need for big data**, 25/03/2020 :
https://www.sas.com/en_us/insights/analytics/big-data-analytics.html

[12] **Characteristics of big data**,25/03/2020, :
https://www.guru99.com/what-is-big-data.html

[13] **Stream processing**, 15/04/2020, :
https://www.ververica.com/what-is-stream-processing

[14] **Apache Storm** , 10/04/2020, :
https://storm.apache.org/

[15] **Apache Spark**,25/03/2020, :
https://www.tutorialspoint.com/apache_spark/apache_spark_introduction.htm

[16] **Apache Spark Architecture**,25/03/2020, :
https://www.edureka.co/blog/spark-architecture/#:~:text=Spark\%20Architecture\%20Overview,Resilient\%20Distributed\%20Dataset\%20(RDD)

[17] **Apache Flink**,26/03/2020, :
https://en.wikipedia.org/wiki/Apache_Flink

[18] **Apache flink ecosystem** , 26/03/2020:
https://www.researchgate.net/publication/333706228_Evaluation_of_distributed_stream_processing_frameworks_for_IoT_applications_in_Smart_Cities/figures

[19] **Big data analytics**,26/03/2020, :
https://searchbusinessanalytics.techtarget.com/definition/big-data-analytics

[20] **Life cycle of big data analytics**,26/03/2020, :
https://www.slideshare.net/Simplilearn/big-data-analytics-what-is-big-data-analytics-big-data-analytics

[21] **Types of big data analytics**,10/04/2020, :
https://imaginenext.ingrammicro.com/data-center/four-types-of-big-data-analytics-and-examples-of-their-use-1

[22] **Drive smart**,10/04/2020, :
http://drive-smart.com/en/

[23] **Aviva drive**,10/04/2020, :
https://www.aviva.co.uk/insurance/motor/aviva-drive/

[24] **Tomtom curfer**,10/04/2020, :
https://www.01net.com/tests/test-tomtom-curfer/le-boitier-qui-veut-vous-aider-a-mieux-conduire-5634.html

[25] **Performance**,10/04/2020, :
https://play.google.com/store/apps/details?id=ie.bluetree.android.incab.performance

[26] **Lambda architecture**, 24/05/2020:
https://www.blue-granite.com/hs-fs/hubfs/Hot\%20Cold\%20Path.png?width=744&height=319&name=Hot\%20Cold\%20Path.png

[27] **Kappa architecture**, 24/05/2020:
www.oreilly.com

[28] **3-tier architecture**,11/04/2020, :
https://d1.awsstatic.com/whitepapers/AWS_Serverless_Multi-Tier_Architectures.pdf

[29] **Kafka** 28/04/2020, :
https://aws.amazon.com/fr/msk/what-is-kafka/

[30] **Bitbucket**, 13/04/2020:
ful.https://www.syloe.com/glossaire/bitbucket/

[31] **Git**, 13/04/2020:
https://git-scm.com/

[32] **Kafka Connect**, 20/05/2020:
https://media-exp1.licdn.com/dms/image/C4D12AQE72Eu9EXHL0w/article-inline_image-shrink_1000_1488/0?e=1594857600&v=beta&t=KQ0Y3CP6gdGBcTet6KyHPohgZA7nLemjE-HcA_WwY04
https://docs.confluent.io/current/connect/index.html

[33] **devops**, *Java Server Pages*, 06/06/2020 :
https://gfi.world/download/14037/DevOps-02.jpg

[34] **Metabase**, 06/06/2020 :
https://logz.io/blog/business-intelligence-tools/

[35] **Scala**, 15/05/2020 :
https://docs.scala-lang.org/tour/tour-of-scala.html

[36] **Google roads API**, 15/05/2020 :
https://developers.google.com/maps/documentation/roads/speed-limits

[37] **Flink windows**, 30/06/2020 :
https://ci.apache.org/projects/flink/flink-docs-stable/dev/stream/operators/windows.html

[38] **Docker**, 21/06/2020 :
https://opensource.com/resources/what-docker

[39] **Jenkins**, 22/06/2020 :
https://www.jenkins.io/

Abstract—
This report describes the stages of design and development of a driver scoring application to represent their behavior through a rating system. Mainly used by fleet managers, this application allows real-time monitoring of scores as well as construction of driver profiles, thus facilitating the monitoring and evaluation of their behavior throughout their journeys. It also allows drivers to self-assess while doing their job.
Key words : driver score, big data, devOps, real-time

ملخص---

يصف هذا التقرير مراحل تصميم وتطوير تطبيق تسجيل السائقين لتمثيل سلوكهم من خلال نظام التصنيف. يستخدم هذا التطبيق بشكل رئيسي من قبل مديري الأساطيل ، ويسمح بمراقبة الدرجات في الوقت الحقيقي وكذلك بناء ملفات تعريف السائقين ، وبالتالي تسهيل مراقبة وتقييم سلوكهم طوال رحلاتهم. كما يسمح للسائقين بالتقييم الذاتي أثناء قيامهم بعملهم.
الكلمات الرئيسية : نقاط السائق، البيانات الكبيرة، ديفوبس

Résumé —
Ce rapport décrit les étapes de conception et de développement d'une application de scoring des conducteurs permettant de représenter le comportement de ceux-ci grâce à un système de notation. Principalement utilisée par les gestionnaires de flottes, cette application permet un suivi en temps réel des scores ainsi qu'une construction des profils des conducteurs facilitant ainsi la surveillance et l'évaluation du comportement de ces derniers tout au long de leurs trajets. Elle permet aussi aux conducteurs de s'auto-évaluer pendant l'accomplissement de leur travail.
Mots clés : score du conducteur, big data, devOps, temps réel